

ClassBuilder Getting Started

Author: Jimmy Venema

Date: Nov 1998

Version: Draft

Table of contents

1. INTRODUCTION	4
2. A QUICK TOUR.....	6
2.1 CREATING A NEW DOCUMENT.....	6
2.2 ADDING A CLASS	9
2.3 ADDING A MEMBER	11
2.4 ADDING A RELATION.....	13
2.5 ADDING A CONSTRUCTOR.....	17
2.6 MANIPULATING ARGUMENTS	20
2.7 ADDING A METHOD.....	21
2.8 SAVING YOUR DESIGN AND GENERATING SOURCE CODE	27
2.9 FINISHING THE PROGRAM.....	28
2.10 GENERATING DOCUMENTATION	32
3. RELATIONS AND LIFETIME OF OBJECTS.....	34
3.1 LIFE TIME	34
3.1.1 Definitions.....	34
3.1.2 Association.....	34
3.1.3 Aggregation	34
3.1.4 Transformations.....	35
3.1.5 The effect on constructors and destructors	35
3.2 SINGLE ASSOCIATIONS	36
3.2.1 Active object methods.....	37
3.2.2 Passive object methods	38
3.3 SINGLE AGGREGATIONS.....	38
3.4 NAMING SINGLE RELATIONS	39
3.5 MULTI ASSOCIATIONS	40
3.5.1 Active object methods.....	42
3.5.2 Iterator class	45
3.5.3 Passive object methods	48
3.6 MULTI AGGREGATIONS	48
3.7 NAMING MULTI RELATIONS.....	50
3.8 STATIC MULTI ASSOCIATIONS.....	51
3.8.1 Active object methods.....	52
3.8.2 Passive object methods	52
3.9 STATIC MULTI AGGREGATIONS.....	52
3.10 NAMING STATIC MULTI RELATIONS	54
4. DIALOG BOX SETTINGS	55
4.1 DATAMODEL.....	55
4.2 CLASS.....	55
4.3 EXTERN CLASS	55
4.4 TYPE	55
4.5 INHERIT	55
4.6 RELATION.....	55
4.7 MEMBER	55
4.8 MEMBER FUNCTION (METHOD)	55
4.9 CONSTRUCTOR	55
4.10 ARGUMENT.....	55
4.11 GROUP.....	55
5. CODE EDITING	56
6. OTHER FEATURES	57
6.1 WRAPPING METHODS FOR MEMBERS WHICH ARE CLASSES	57

6.2	ADDING VIRTUAL FUNCTIONS TO OVERRIDING CLASSES.....	57
6.3	ADDING 'ISCLASS' METHODS.....	57
6.4	SUPPORT FOR MULTI THREADED APPLICATIONS	57
7.	WRITING DOCUMENTATION.....	59
8.	SERIALIZING YOUR DATA MODEL.....	60
9.	DYNAMIC REPLACEMENT OF OBJECTS.....	61
10.	SOME 2.0 USAGE REMARKS	63

1. Introduction

In an object oriented design often many different relations between the various objects exist. The existence of a relation between objects has consequences for the members and methods of their classes. It is a tedious and error-prone work to model these relations at this relatively low level. To gain experience if a certain data model is a usable one, a lot of implementation effort has to be invested. Changing the data model into a better one, often implies an enormous amount of re-work. Therefore, the original design is just patched in the implementation phase to make it workable. After the implementation phase the actual relations between the different objects are unclear. They have to be extracted from the class members and method source code. The same relation type can be implemented in various ways, which hinders this extraction also.

The above observations lead to the desire for a tool in which the various relations between the objects of the different classes can be specified. The output of the tool would be C++ source code files that implement the specified classes and their relations. It should further be possible to add and manage other members and methods in an easy manner as well. ClassBuilder is such a tool and is described in this document. Other features of the tool are:

- Code generation for Find methods on relations and Get/Set methods for members.
- Wizards to insert code, like iteration loops and navigation paths.
- Code synchronization with compile environment (changes to code of methods are read back in).
- Full code generation for serializing the complete data model when used together with MFC. (The tool is build with itself and uses this feature to save and read).
- Drag and drop support for many things.
- Documentation generation in both HTML and RTF format.
- Reading of class diagrams from Visio 4 or higher by using a supplied stencil.

The document is structured in the following way. First there is a quick tour chapter to get you familiar with the basic workings of the tool. Chapter 3 gives an explanation about relations and their effect on the lifetime of objects. This chapter should give you enough background to understand the behaviour of the generated code. Chapter four deals with the different dialogs and their settings in detail. Chapter 5 explains the code edit facilities. In chapter 6 various handy things like 'Add virtual methods' and 'Add IsClass methods' are described. Chapter 7 explains how you can import information from class diagrams made in Visio. Chapter 8 covers how to generate documentation. Chapter 9 is about serialization support and how you can use it in the MFC document view architecture. Chapter 10 explains the dynamic replace feature.

In this document we use Visual C++ 4.2 as compiler. The examples should work in other compiler environments as well. One restriction is that some examples make use of MFC functionality, if this is the case we state that explicitly. To make things easier just a few words about the notation used.

Key	A key you should press.
Alt+A	An example of two keys to be pressed simultaneously.
“Text”	Text to type in literary, the quotes are not part of the string.
<i>File→New</i>	A navigation path through the menu bar. In this case select ‘File’, followed by clicking the ‘New’ menu item.
 <i>Add→Class</i>	A navigation path via the pop up menu enabled by clicking the right mouse button. In this case pop up the menu, select ‘Add’, followed by clicking the ‘Class’ menu item.
 <i>Add Relation</i>	Click the ‘Add Relation’ tool bar button.
<i>(Add→Class, Ctrl+Alt+C)</i>	Alternative selection paths to trigger action.
 QuickTour	Select item ‘QuickTour’ in tree view.

2. A quick tour

In this tour we make a small working program. It is not a meaningful program by itself, but its creation process should be educational. We are going to define two classes, a relation between the two classes, a few methods and a member. Then we generate the code, finish the code, compile and run the program.

2.1 Creating a new document

First you have to define a new document, this document contains information about classes, relations between classes, inheritance, members, methods, etc. All this information combined we call a data model. To create a new document suited for this quick tour follow the steps below.

1. Start the ClassBuilder tool.
2. Select  New (File→New, **CTRL+N**), see Figure 2-1.

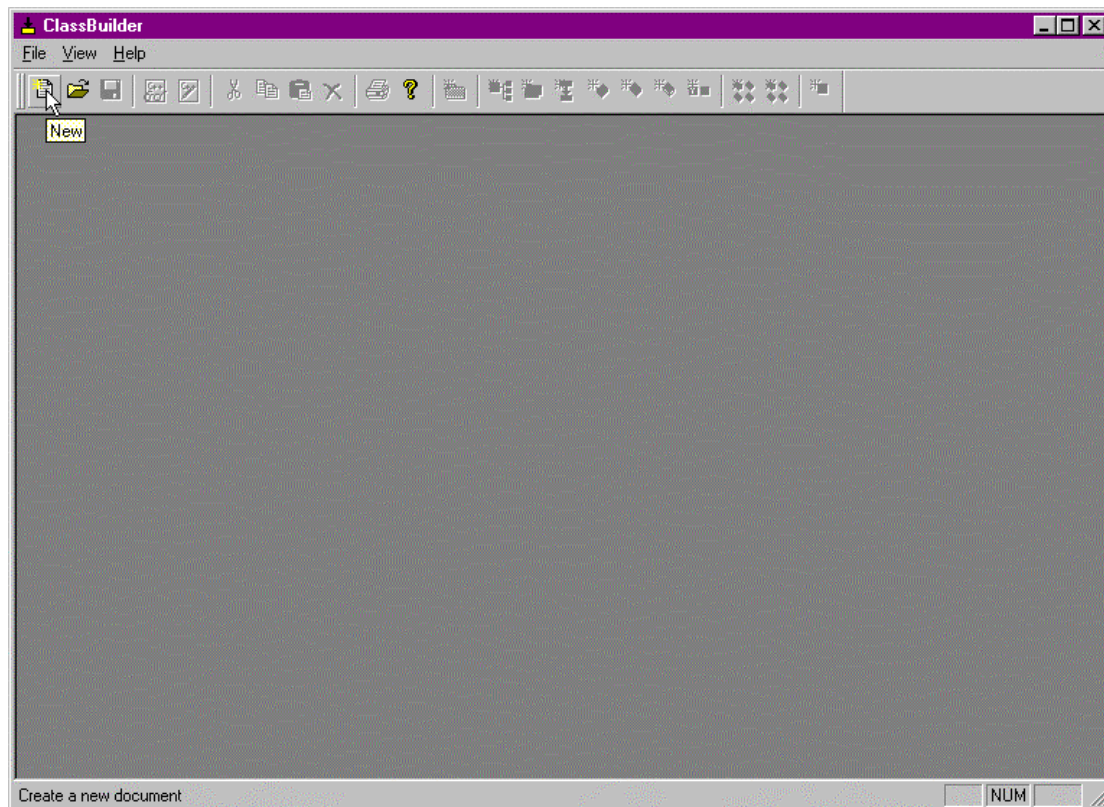


Figure 2-1: The ClassBuilder window, while selecting the 'New' tool bar button.

A dialog should pop up, see Figure 2-2.

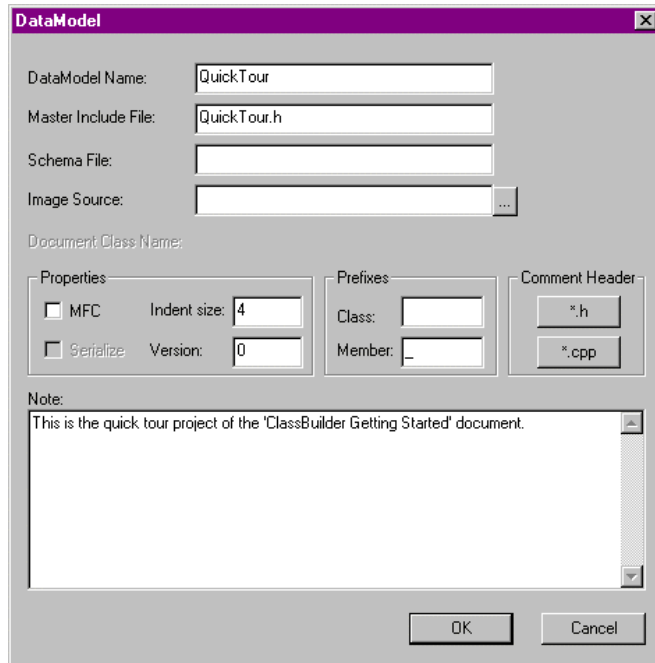


Figure 2-2: The DataModel dialog filled in.

3. Fill the dialog box as shown above and click on 'OK'. You can omit the entries 'Schema File', 'Image Source' and the prefix of 'Class'.

Your screen should look the same as, Figure 2-3. Now you have created an empty data model named 'QuickTour'. Below the QuickTour item all classes that are part of this data model are placed. Apart from the QuickTour item there are two other items visible in the tree. Below the Extern Classes item all classes are placed which are not part of your data model. No code is generated for these classes, but they are needed for referencing purposes. For example if you would like to inherit from an existing class, like the ones in MFC. Below Other Types all other non class types are stored. You find here things like 'int', 'char', 'double', etc. They are also needed for referencing purposes.

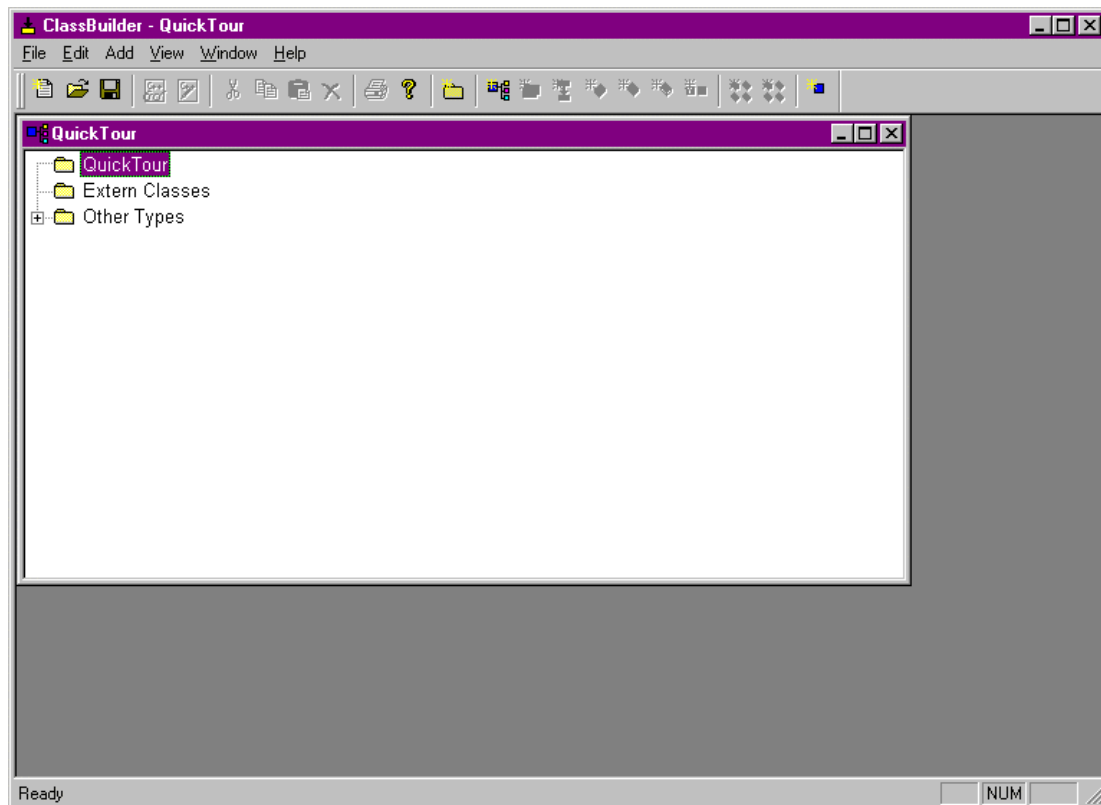


Figure 2-3: Tree view window to navigate data model.

Next you open an additional window to view the tree and rearrange these view windows.

4. In the tree window with title 'QuickTour' select Other Types.
5. From the menu select *Window→New Sub Window* (*New Sub Window*).

A new window is created with as title 'QuickTour:\Other Types' and is made active. Only a portion of the total tree can be viewed in this new window, the navigation path of where its tree starts is included in the title. The title of the original window changed from 'Quick Tour' to 'Quick Tour:1', this new title reflects that more than one view is active on the same ClassBuilder document.

6. Expand the Other Types in the new window by either clicking on its symbol or by pressing **+** on the numeric key pad (this is standard Windows behaviour).
7. Make the original window active by clicking on it.
8. Select from the menu *Window→Tile*.

Compare your result with that of Figure 2-4 shown below.

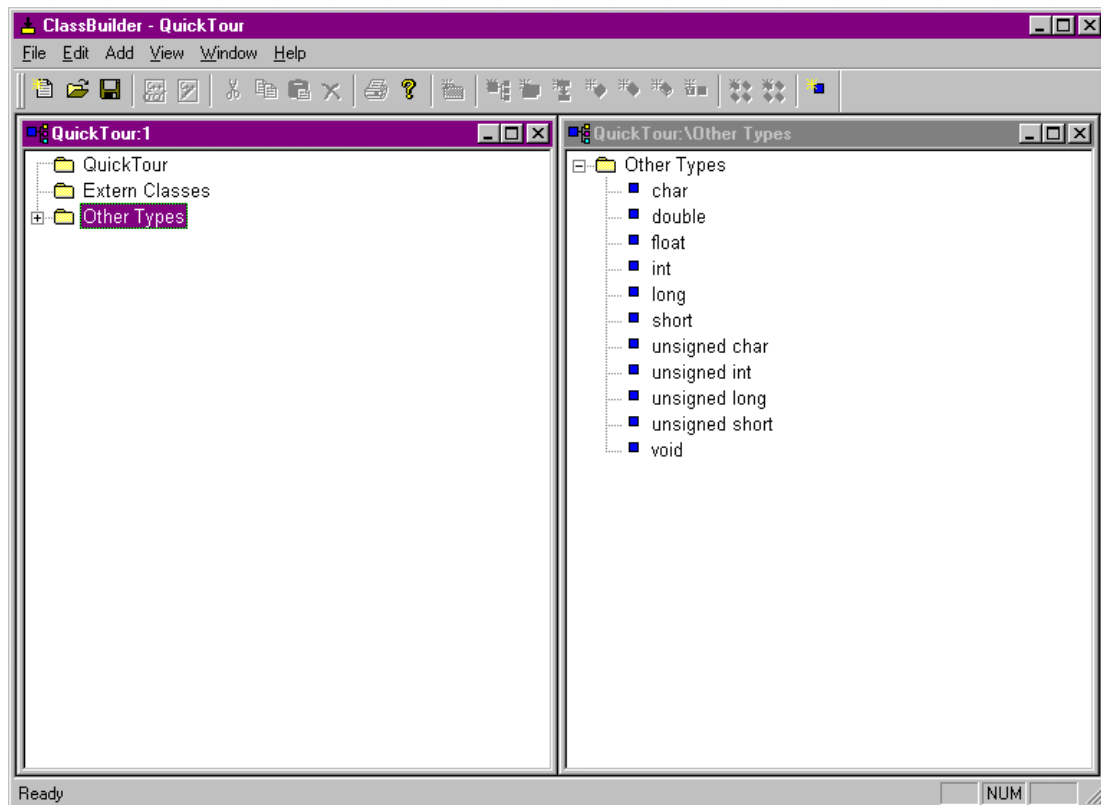


Figure 2-4: Two views on the same data.

2.2 Adding a class

During this part of the tour you add the classes 'A' and 'B' to your data model.

1. Select  QuickTour in the left tree window.
2. Select with the toolbar  Add Class (Add → Class,  Add → Class, **Ctrl+Alt+C**), see Figure 2-5.

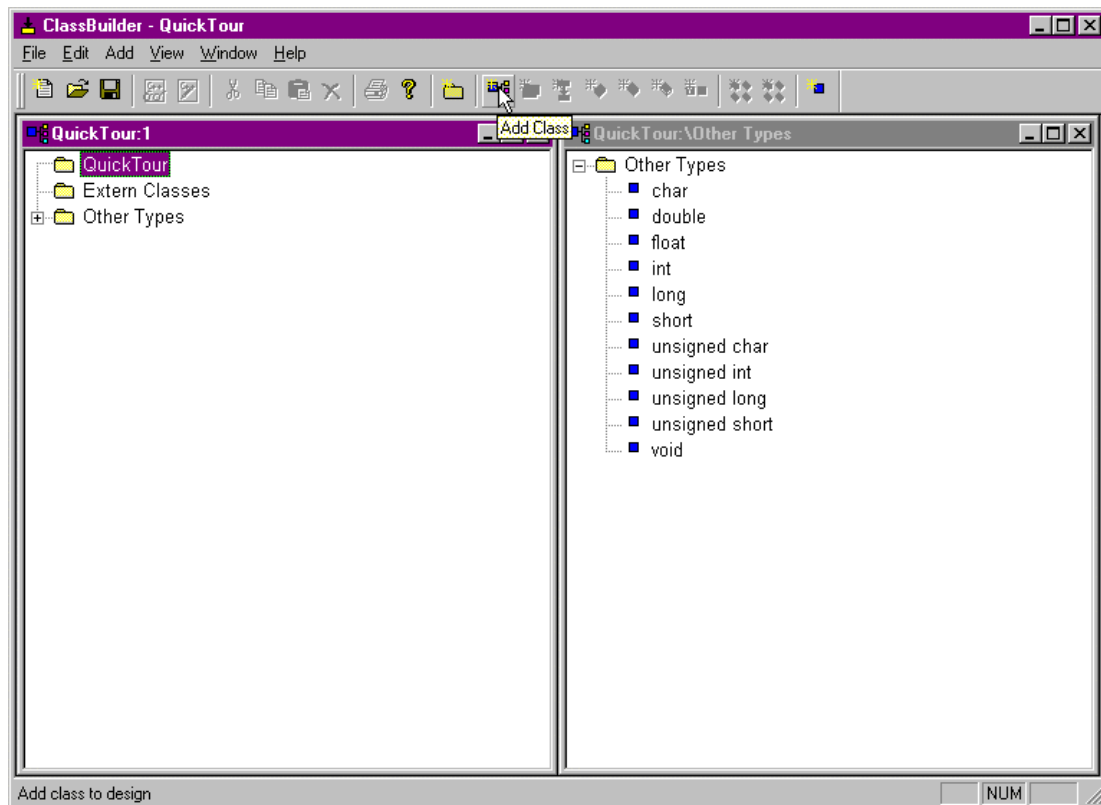


Figure 2-5: Adding a class to the QuickTour data model.

The Class dialog box appears, see Figure 2-6.

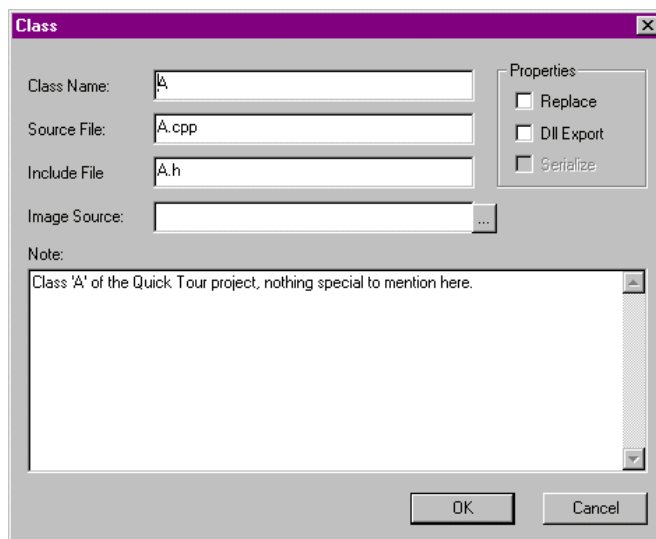


Figure 2-6: The Class dialog box.

3. Fill your dialog box as shown in Figure 2-6, to create a class 'A' with as associated source files 'A.h' and 'A.cpp'.

Notice that the source file names are derived automatically from the class name. You can use other file names as long as they are unique.

4. Repeat the steps 1, 2 and 3 to create a class named 'B', use the default names for its source files. For the 'Note' field you can type in the following text "Class 'B' of the Quick Tour project, nothing special to mention here."

- Expand the tree items A and B, by clicking on the plus sign left of it.

Your screen should look similar to Figure 2-7 as shown below.

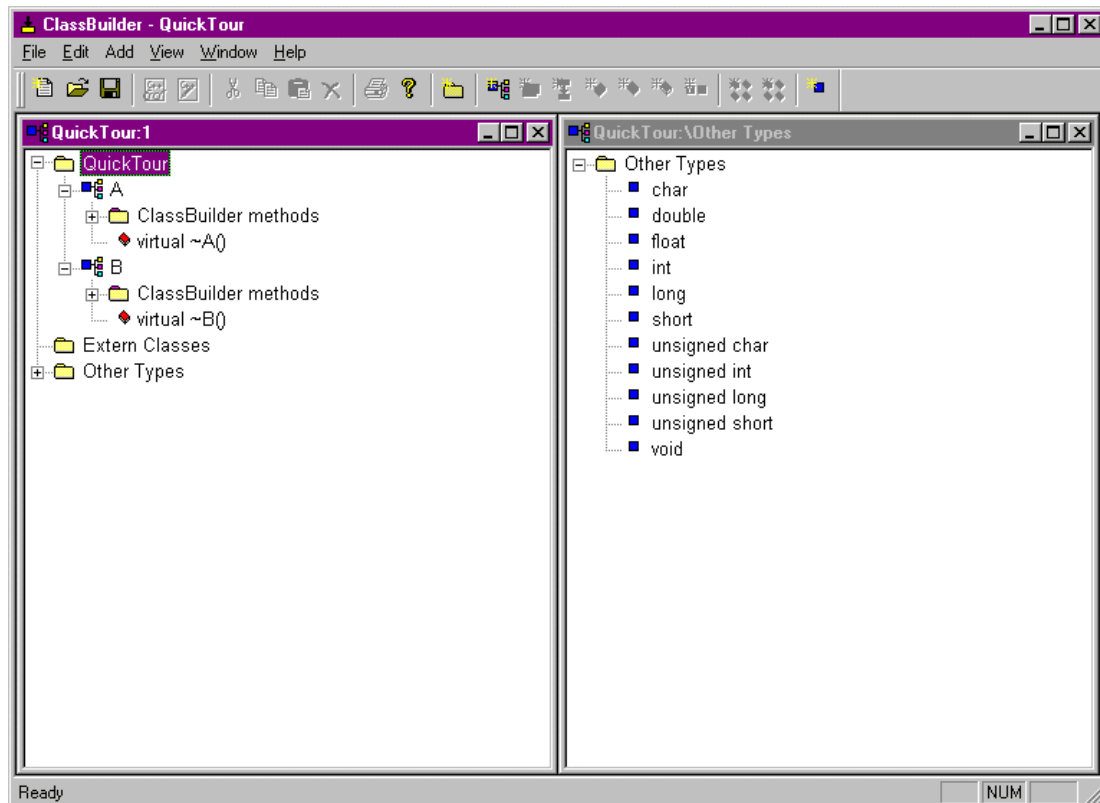


Figure 2-7: Overview of the QuickTour data model, with two classes.

For each class added a destructor is also added, plus in this case two methods, which are placed below the ClassBuilder methods item. Lets take a look at these generated methods.

- Expand ClassBuilder methods below A by clicking on its symbol left of it.

Now the two methods void ConstructorInclude() and void DestructorInclude() are visible.

The ConstructorInclude() method must be invoked first in (almost) every constructor. Depending on the defined relations, several different actions must be taken when an object is created. The ConstructorInclude() method takes care of that. The actual argument list of the ConstructorInclude() method also depends on the defined relations.

The DestructorInclude() method must be invoked in the destructor. Depending on the defined relations several different actions must be taken when an object is deleted. The DestructorInclude() method takes care of that.

- Collapse the ClassBuilder methods item by clicking on its symbol.

2.3 Adding a member

Now you will learn how to add a member to a class by using drag and drop. Also you will see how easy it is to add methods which get or set a member.

1. Select  int in the right tree window and drag it while holding down the **Ctrl** key.
2. Drop it onto the class item  B in the left window, which gets high lighted.
3. A dialog box is opened, fill in as shown below in Figure 2-8 and click on the 'OK' button.

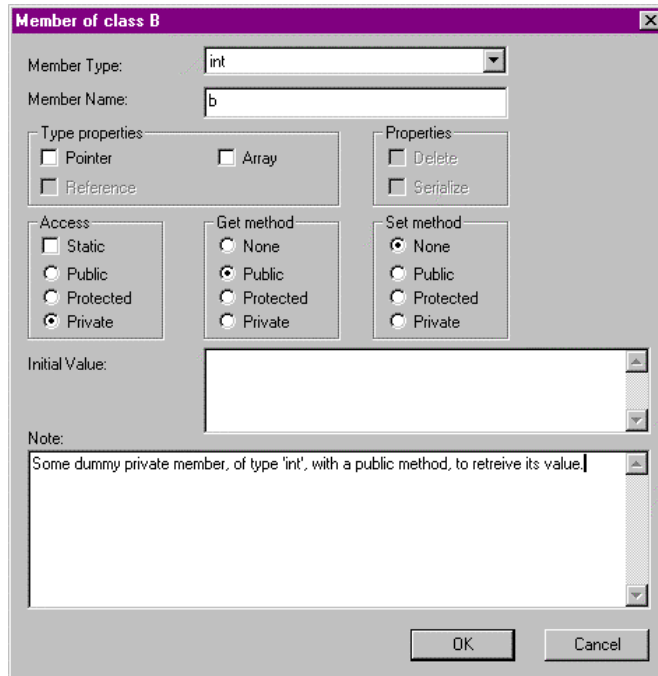


Figure 2-8: The member dialog box.

Now you have added a member 'b' of type 'int' for class 'B' and a public method retrieving the value of 'b'. To add a method for setting the value of '_b', just select another radio button as 'None' in the 'Set method' group.

4. Expand the  int _b item.

Your screen should look as shown in Figure 2-9. Notice that although we defined the name of the member as 'b' in the dialog, it is shown in the tree as '_b'. This is because the prefix for members is defined as '_', which is the default. This setting can be changed in the DataModel dialog, see Figure 2-2. To pop up this dialog box just double click the  QuickTour item. When this prefix is changed everything is updated accordingly.

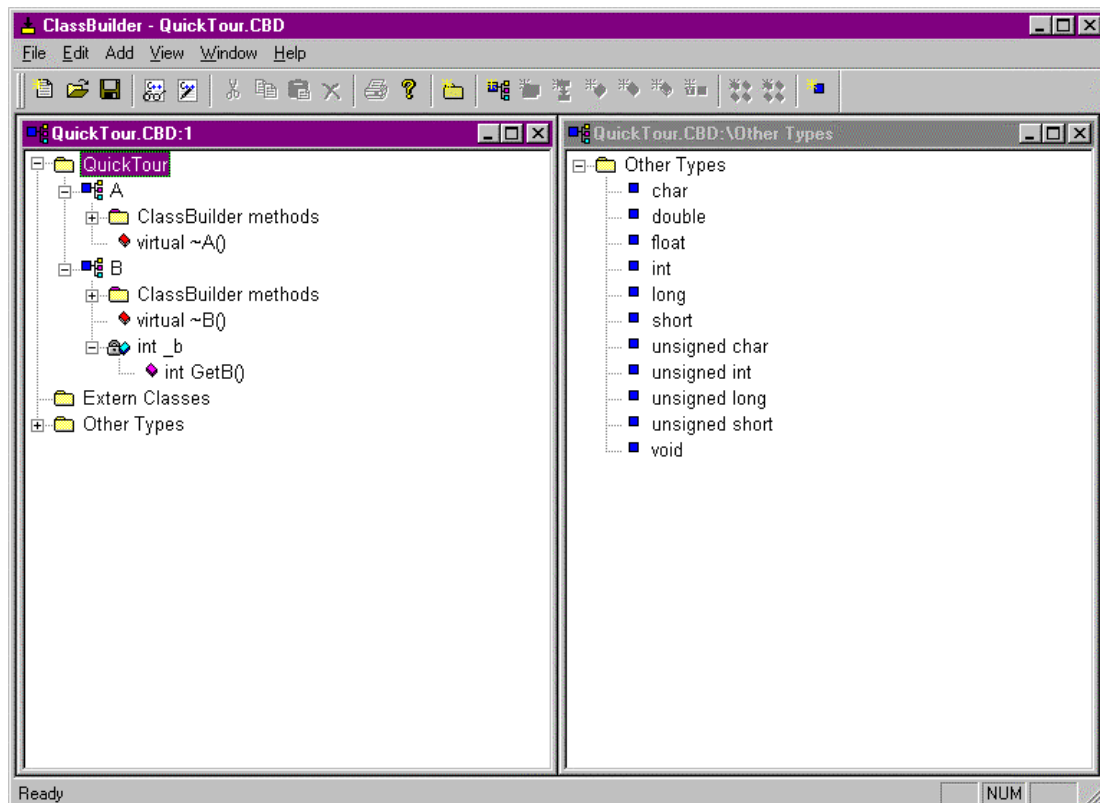


Figure 2-9: Member '_b' and its retrieve method 'GetB()' are added to the tree.

5. Double click on `int GetB()` to show its code, the resulting window is shown in Figure 2-10.

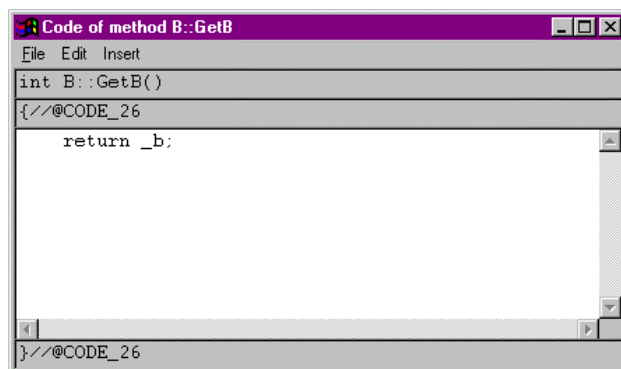


Figure 2-10: The code of method B::GetB().

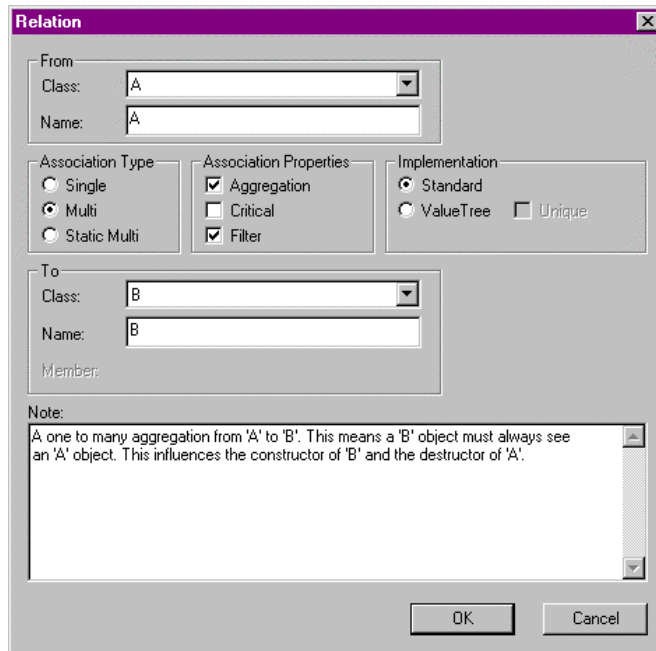
6. Select *File*→*Close*, to close this window after inspecting it.

Note that only the white areas can be edited, be sure to only edit these parts when using an other editor on the source code. When using another editor strings like “`//@CODE_26`” are used by ClassBuilder to recognize the parts it has to read in. It can be that a different number is used in your case.

2.4 Adding a relation

Next we are going to define a one to many aggregation between class ‘A’ and class ‘B’. This means an object of class ‘A’ can see zero or more ‘B’ objects and a ‘B’ object must always see an ‘A’ object.

1. Select class item  A in the left tree window.
2. Select  Add Relation (Add → Relation,  Add → Relation, **Ctrl+Alt+R**).



The 'Relation' dialog box is shown with the following settings:

- From:**
 - Class: A
 - Name: A
- Association Type:**
 - ☐ Single
 - ☒ Multi
 - ☐ Static Multi
- Association Properties:**
 - ☒ Aggregation
 - ☐ Critical
 - ☒ Filter
- Implementation:**
 - ☒ Standard
 - ☐ ValueTree
 - ☐ Unique
- To:**
 - Class: B
 - Name: B
 - Member:
- Note:**

A one to many aggregation from 'A' to 'B'. This means a 'B' object must always see an 'A' object. This influences the constructor of 'B' and the destructor of 'A'.

Buttons: OK, Cancel

Figure 2-11: The relation dialog for a one to many aggregation from 'A' to 'B'.

3. Fill the popped up dialog window as shown in Figure 2-11 and click on the 'OK' button.
4. Expand item  B, below item  A.
5. Expand item  Relation methods, below item  B.
6. Expand item  A, below item  B.
7. Expand item  Relation methods, below item  A.

Compare your screen with that of Figure 2-12 shown below.

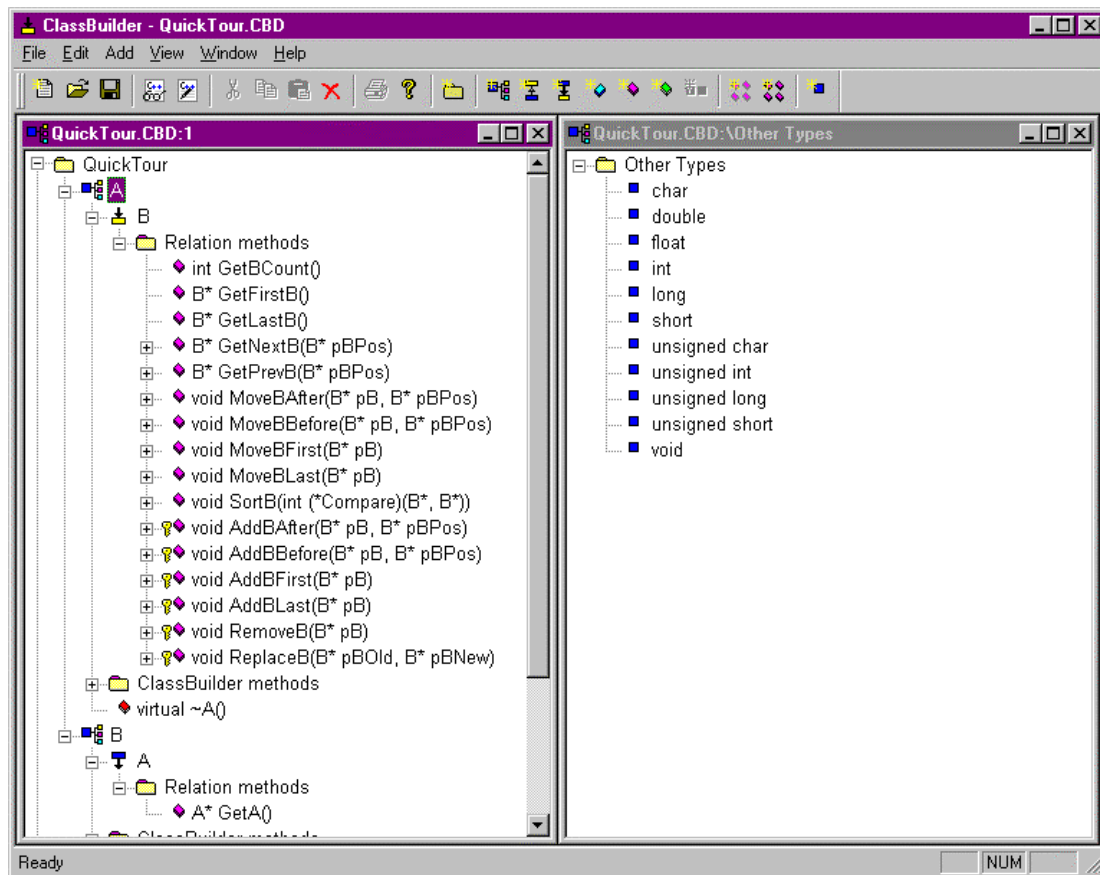


Figure 2-12: Overview of all methods related to the relation between 'A' and 'B'.

Note the amount of methods generated for the added relation; all these methods including their implementation are generated by ClassBuilder. Note also where these methods are placed in the tree, they are separated from the other methods to keep overview and to ease navigation. Normally you would not expand the 'Relation methods' items, because the methods inside it are fully predictable.

8. Collapse both Relation methods items.

Often when there is a one to many relation, there is the need to find one object among many. ClassBuilder makes it very easy to make such a find method. Let us add a find method at class 'A' to find a 'B' object where its '_b' member has a certain value.

9. Select relation item B, within class item A.
10. Click on the toolbar button Add Member Function (Add→Member Function, Add→Member Function, **Ctrl+Alt+F**).

A window with as title 'Find Method' appears. Because the B item was selected when adding a method, ClassBuilder assumes you wanted a find method operating via the selected relation.

11. Select the ->GetB() item in the right tree pane and click on the '<->' button.

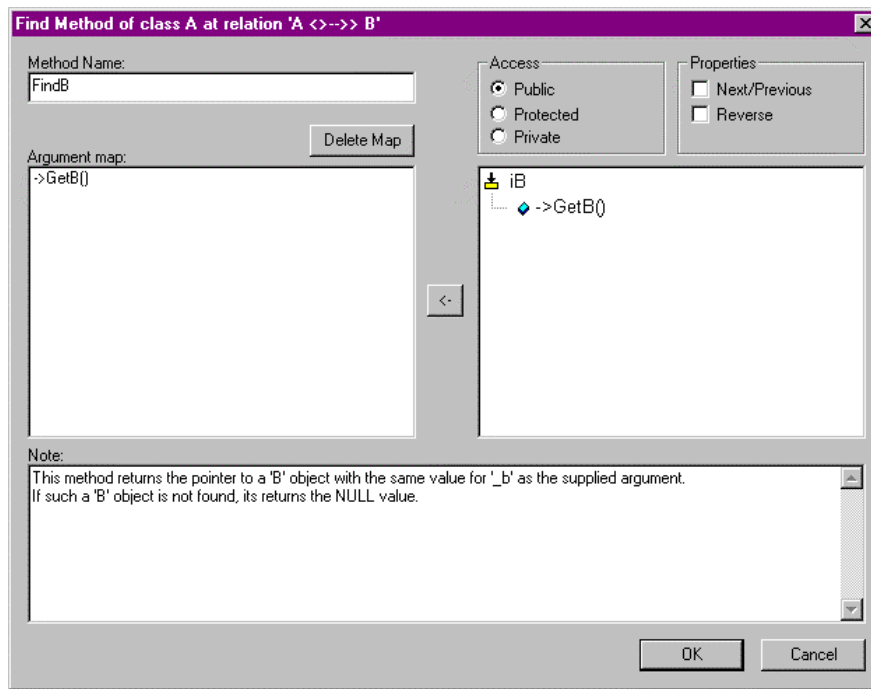


Figure 2-13: The 'Find Method' dialog, find a 'B' object with a certain value for its '_b' member.

The  iB item in the right tree pane symbolizes the iterator used in the find method. The find method iterates all 'B' objects, what we have to define here are the criteria a 'B' object must fulfill. Below the  iB item all possible items are shown which can be part of the criteria. In this case only the 'GetB()' retrieve method of the '_b' member is shown. In case a member has a retrieve method, this dialog has the preference of showing its retrieve method. This retrieve method is also used in the code of the find method. To show that it is a retrieve method, the cyan icon of a member is used in front of the retrieve method name.

In the left pane all selected criteria for the current find method are shown. Each selected criterion is mapped to an argument where it is compared against. The actual arguments itself aren't shown in this dialog.

12. Fill in some text at the 'Note' field and click on the 'OK' button if your window look the same as Figure 2-13 shown above.

Notice that an argument 'b' of type 'int' is generated. In case you are not happy with its name, just wait till the tour visits 'Manipulating arguments'.

13. Double click on the  B* FindB(int b) item in the tree window to look at the generated code, see Figure 2-14. Be aware that the '//@CODE_69' string may have a different value in your case.

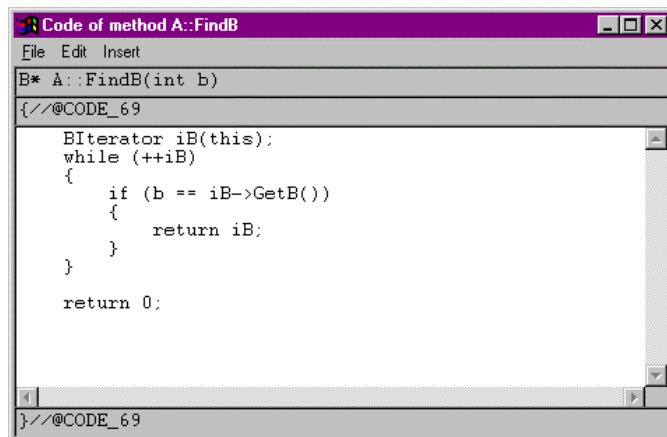


Figure 2-14: The generated code for the A::FindB() method.

The code shows you how to iterate a one to many relation and how the criteria are checked. The code for the iterators on a relation are also generated. For more information about iterators see section 3.5.2 'Iterator class'.

14. When you are ready inspecting the code, close the code window.

2.5 Adding a constructor

Now it is time to give class 'B' and class 'A' a constructor. Be aware that the constructors generated are just an initial guess, based on the information present. That is the reason we waited till the relation and member were added, now the guess of ClassBuilder is more accurate. If you're not happy with the generated constructor, just change it.

1. Select the  class item in the tree.
2. Click on the  Add Constructor button in the tool bar (Add → Constructor,  Add → Constructor, **Ctrl+Alt+N**).

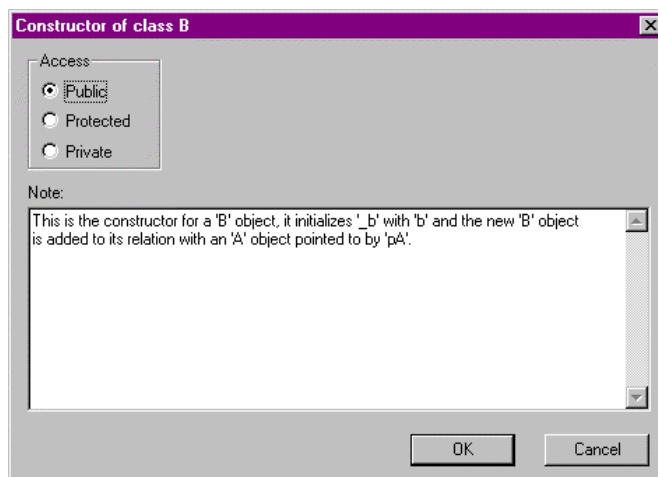


Figure 2-15: The constructor dialog.

3. Fill in the dialog box as shown in Figure 2-15 and click on the 'OK' button.
4. Repeat steps 1, 2 and 3 for class 'A', fill in as note text "This is the constructor for an 'A' object." or any other text you like.
5. Expand item  B(A* pA, int b) by clicking on the plus sign to the left of it.

Your screen should look similar to the one shown in Figure 2-16.

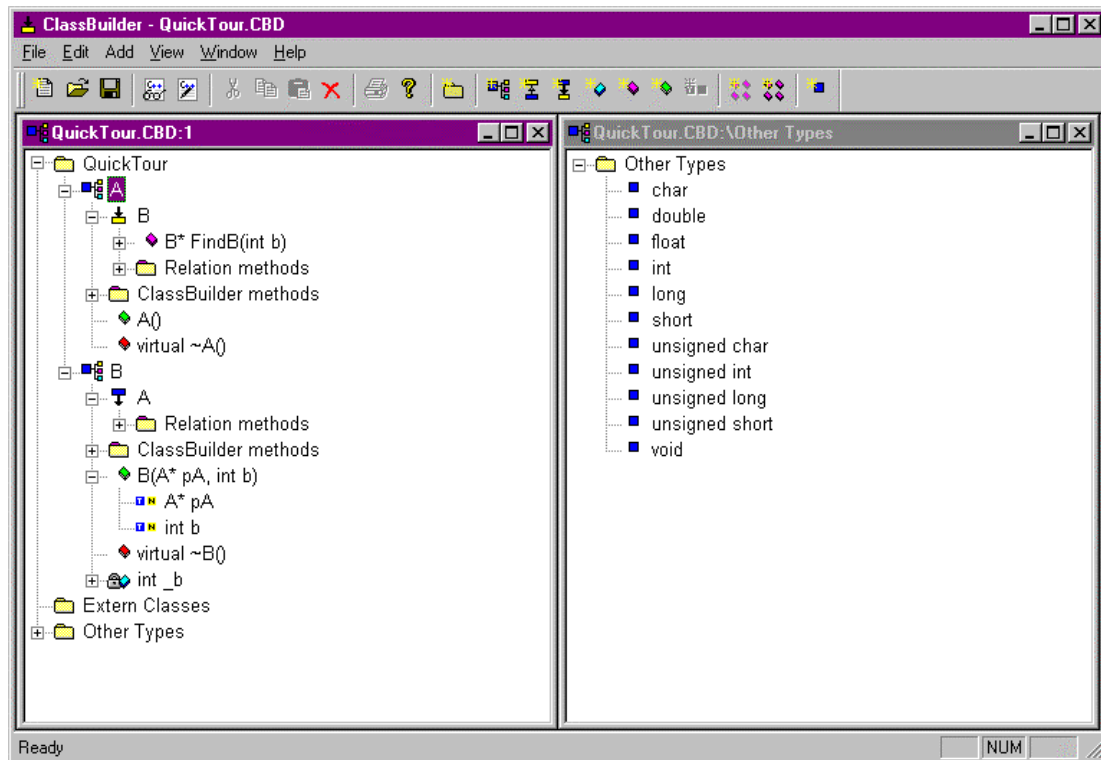


Figure 2-16: The ClassBuilder screen after adding the constructors.

6. Double click on the constructor item `B(A* pA, int b)` to investigate its code, see Figure 2-17.

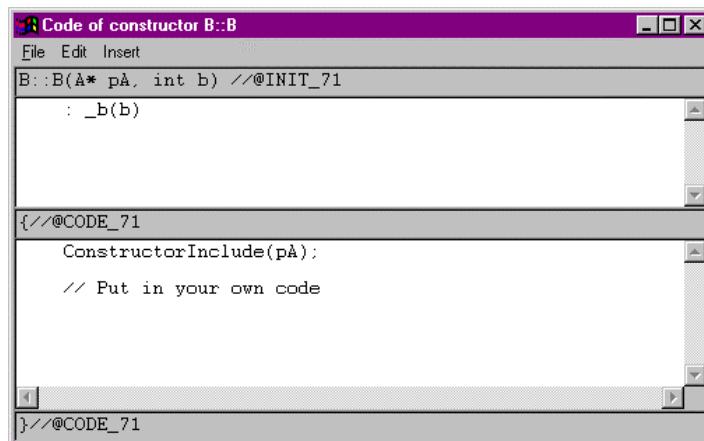


Figure 2-17: The initial code for constructor `B::B(A* pA, int b)`.

The generated code for a constructor when newly created is just initial, you can alter it to your liking, it must however include a call to the 'ConstructorInclude()' method. The 'ConstructorInclude()' method makes sure that the constructor has the correct behaviour with respect to the defined relations. The same holds true for the destructor, it must include a call to the 'DestructorInclude()' method to have a correct behaviour.

The constructor of class 'B' has as argument a pointer 'pA' to an 'A' object. This is because its relation with 'A' is an aggregation which means a 'B' object must always see an 'A' object. This behaviour is forced into the constructor, and is implemented

via the above mentioned 'ConstructorInclude()' method which is automatically generated and updated by ClassBuilder.

Note that member '_b' is initialized with argument 'b'. This is the default behaviour for generating a constructor, when a member has no initial value. When a member has an initial value, then the member is initialized with this value and no argument is generated. To try this out, carry out the following steps.

7. Double click on the  int _b item in the tree to open the member dialog box.
8. Fill in "0" at the 'Initial Value' field and click on the 'OK' button.

You are free to type in any text in the 'Initial Value' field, so be careful, errors are first spotted during compilation.

9. Click on the  Add Constructor button in the tool bar.
10. Click on the 'OK' button of the constructor dialog.

Now a new constructor is generated without the 'b' argument.

11. Double click on  B(A* pA) to pop up its code window, see Figure 2-18.

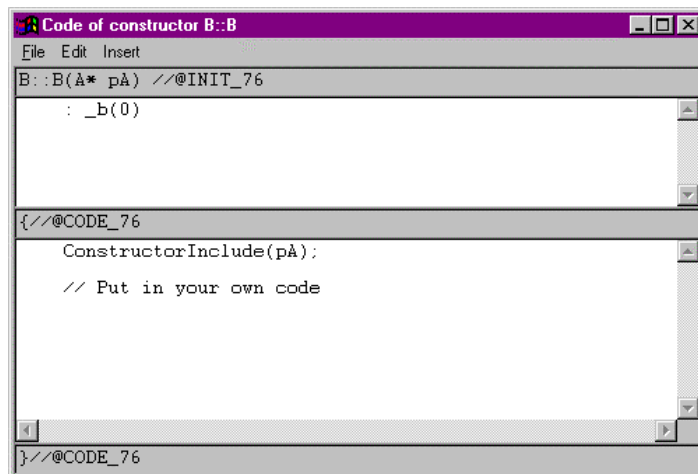


Figure 2-18: The constructor of class 'B', with '_b' initialized at '0'.

Now member '_b' is initialized with '0' and no argument is needed and thus generated. Once a constructor is generated it isn't updated when adding members or changing relations. You either create a new constructor and throw away the old constructor, or update the existing one. For updating there is some help in the code editor, you can regenerate the initialize part or the body part of the code. To trigger this action use the menu entries *File*→*Regenerate Code* or *File*→*Regenerate Init* of the code editor. The interface isn't altered when using this regeneration.

12. Close the code window of the constructor.
13. Select the constructor item  B(A* pA) and press the **Delete** key (*Edit*→*Delete*,  Delete).

The following message box appears, see Figure 2-19.

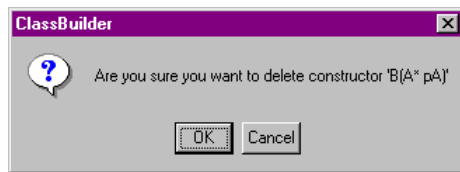


Figure 2-19: Message box verifying the delete action.

14. Click on the 'OK' button, we don't need this constructor, for the rest of this tour.

2.6 Manipulating arguments

In this part of the tour we show you how to manipulate the order of arguments and how to change an argument's name. Make sure the constructor item $\diamond B(A^* pA, \text{int } b)$ is expanded so its arguments items are visible.

1. Select the $\text{int } b$ argument of constructor $\diamond B(A^* pA, \text{int } b)$, keep the left mouse button pressed and start dragging without using the control key (otherwise we would be copying this argument).

Items of the tree are highlighted if you can drop the dragged item there.

2. Drop the $\text{int } b$ drag item on the highlighted $\diamond B(A^* pA, \text{int } b)$ constructor item.

Note that the argument order of the constructor has changed.

3. Select the $\text{int } b$ argument again, drag it to the $A^* pA$ argument and drop it there.

The original order is restored now. See how easy it is to change the order of arguments, no more editing of source and include files, just drag and drop.

4. Double click the $\text{int } b$ argument of constructor $\diamond B(A^* pA, \text{int } b)$.

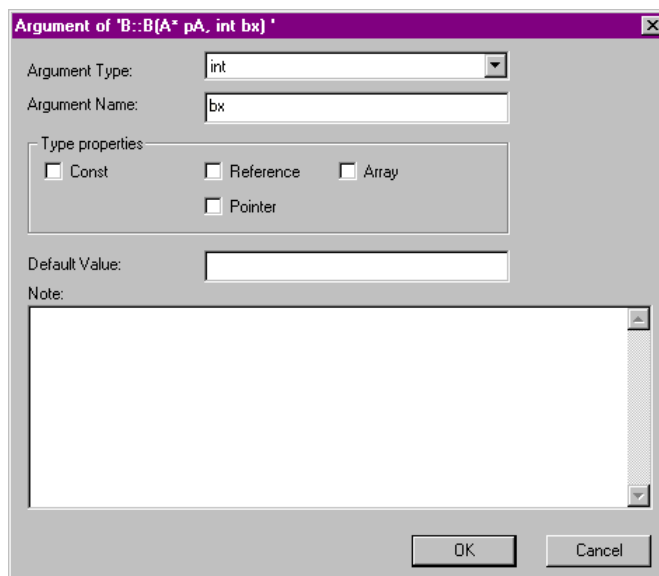


Figure 2-20: Changing the argument name from 'b' to 'bx'.

5. A dialog box appears, change the argument name from 'b' into 'bx', as shown in Figure 2-20 and click on the 'OK' button.
6. Double click on constructor $\diamond B(A^* pA, \text{int } bx)$ and inspect its code, see Figure 2-21.

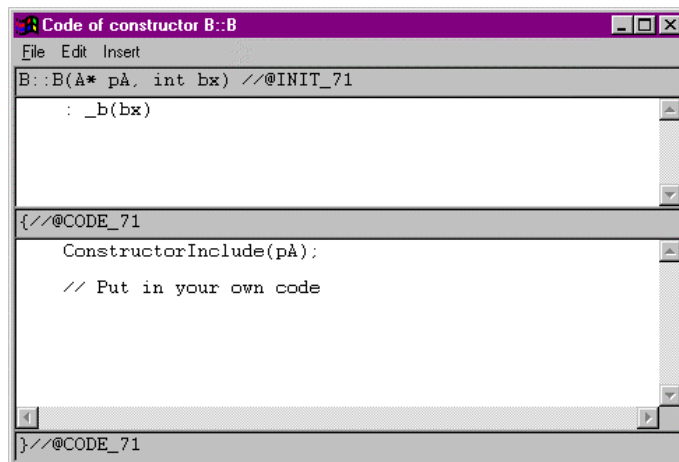


Figure 2-21: The code of the constructor, after changing the name of its argument.

Note that the code of the constructor is altered to reflect the change in the argument name. This automatic update of code also applies when changing type, class or method names. This feature is even more appreciated when changing the name or argument list of a virtual method. All overriding virtual methods are updated accordingly. Ever added an argument to a virtual function, which was overridden many times?

7. Close the code window and change argument 'bx' back to 'b'.
8. Collapse the constructor item  B(A* pA, int b), by clicking on its  symbol.

2.7 Adding a method

In this section we add a public method 'IsOdd()' to class 'B' and a static public method 'Test()' to class 'A'. The 'IsOdd()' method returns a non-zero value if the '_b' member has an odd value and a zero value otherwise. The 'Test' method does what it names suggest, it tests the classes 'A' and 'B'. While making these methods we learn more about the code insert feature.

1. Select class item  B in the left tree window.
2. From the menu select *Add→Member Function* ( *Add Member Function*,  *Add→Member Function*, **Ctrl+Alt+F**)
3. A method dialog box appears, fill it in as shown in Figure 2-22 and click on the 'OK' button. Make sure you didn't forget to set the access to 'Public'.

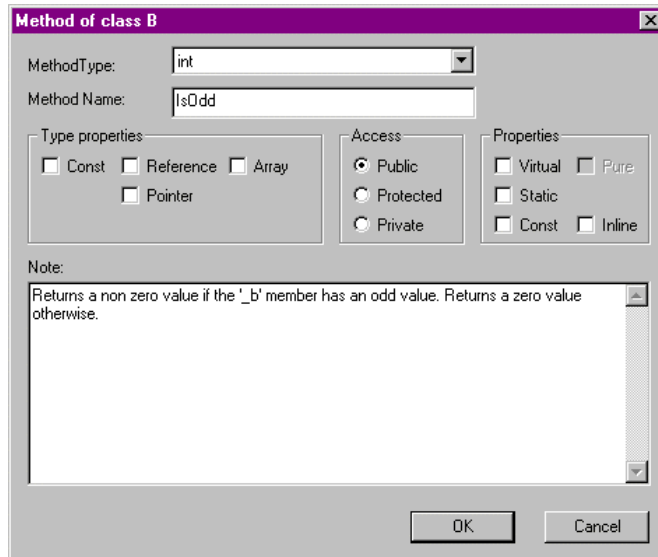


Figure 2-22: The method dialog for 'B::IsOdd()'

Note if you select the  `int IsOdd()` method item and then do a single click (not too fast after selecting it!!) or press the enter key after selecting  `int IsOdd()`. Then the dialog box of Figure 2-22 is displayed again and its fields can be altered.

4. Double click on the  `int IsOdd()` method item and edit the code as in Figure 2-23.

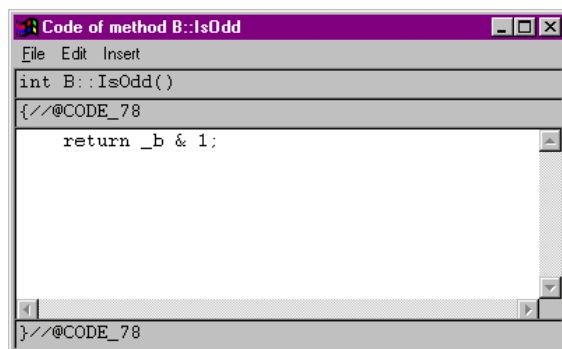


Figure 2-23: The code window of 'B::IsOdd()'

5. Close the code window.
6. A message box pops up asking if you want to save the changes, click on the 'Yes' button.

The 'IsOdd()' is finished, we continue with the static 'Test()' method and use the code insert feature for editing its code.

7. Select class item  A.
8. Click on the  *Add Member Function* button in the tool bar.
9. Make the contents of the displayed dialog the same as the one shown in Figure 2-24.

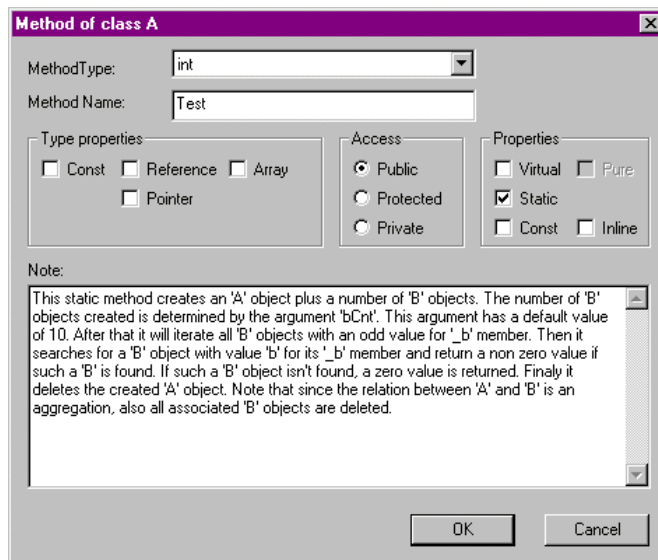


Figure 2-24: The method dialog for 'A::Test()'

10. Check if the access is 'Public' and if the property 'Static' is checked, click on the 'OK' button.
11. Select the  static int Test() method item in the left tree window.
12. Click on the  Add Argument button on the tool bar. Or alternatively drag, while holding down the Ctrl key, the  int type item from the right window, to the left window and drop it on the  static int Test() method item.
13. A dialog box pops up, fill it in as shown in Figure 2-25 and click on the 'OK' button.

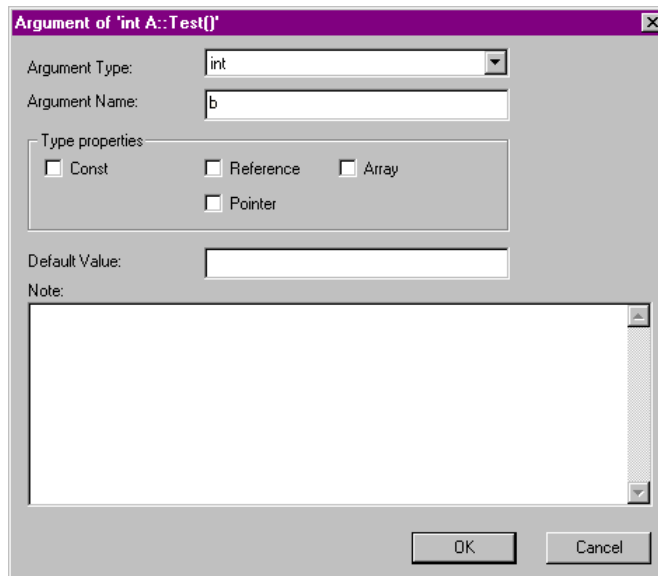


Figure 2-25: Dialog to add argument 'int b' to method 'A::Test()'

14. Repeat steps 11 and 12.
15. Fill in the popped up dialog as shown in Figure 2-26 and click on the 'OK' button.

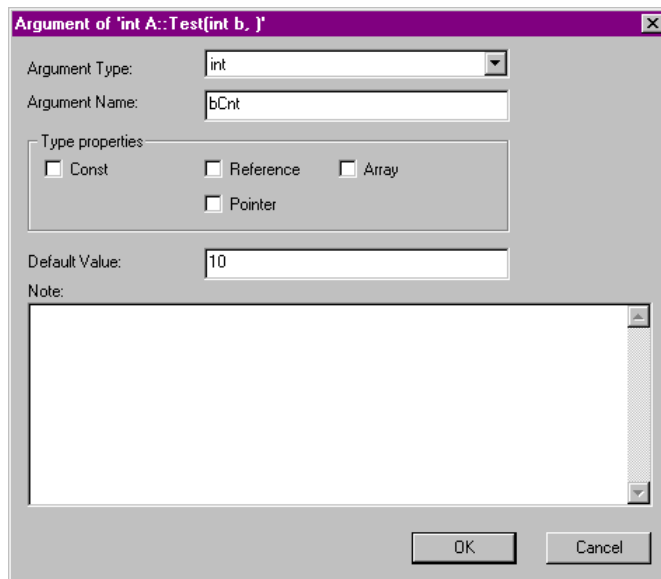


Figure 2-26: Dialog to add argument 'int bCnt = 10' to method 'A::Test()'

Your screen should look like Figure 2-27.

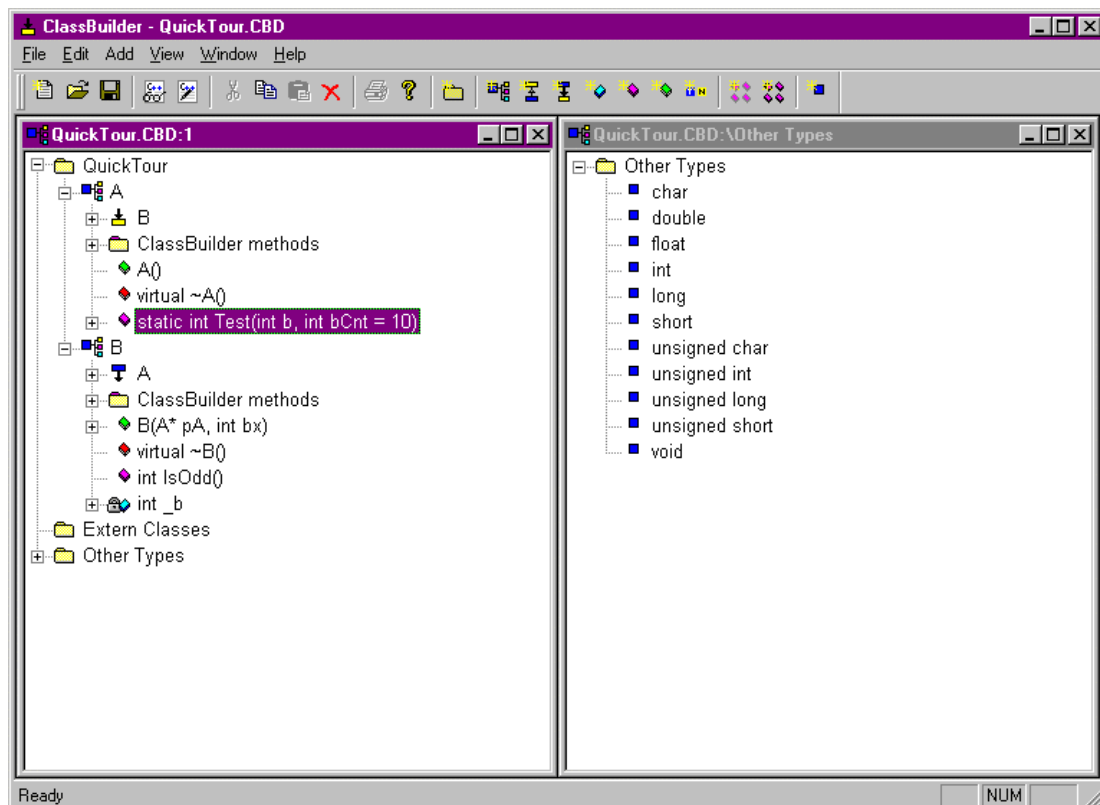


Figure 2-27: Both methods added to the 'QuickTour'.

16. Double click on method item `static int Test(int b, int bCnt = 10)` to start the code editor.
17. First edit the code as shown below in Figure 2-28.

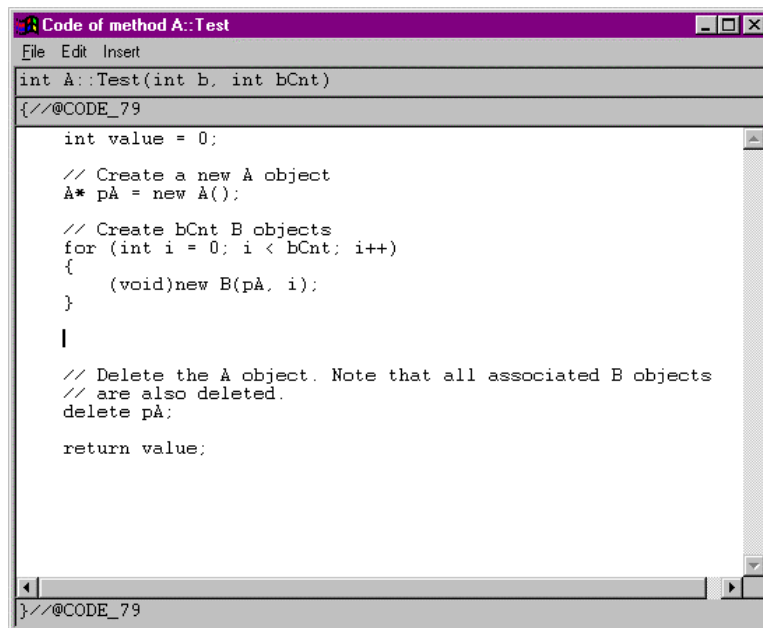


Figure 2-28: The code of method 'A::Test()', before inserting iterator.

Note that selected text can be indented to the right or left by pressing the **Tab** key or by pressing **Shift+Tab**. Selected text can be reformatted by pressing **Alt+F8**.

18. Position the cursor as shown in Figure 2-28.

19. Press the right mouse button and select **Insert → Iterator** from the pop up menu.

A dialog box appears, fill it in as shown in Figure 2-29.

20. Select 'pA' from the left tree pane, 'B' from the middle pane and 'IsOdd' from the right pane.

21. Click 'OK' when finished.

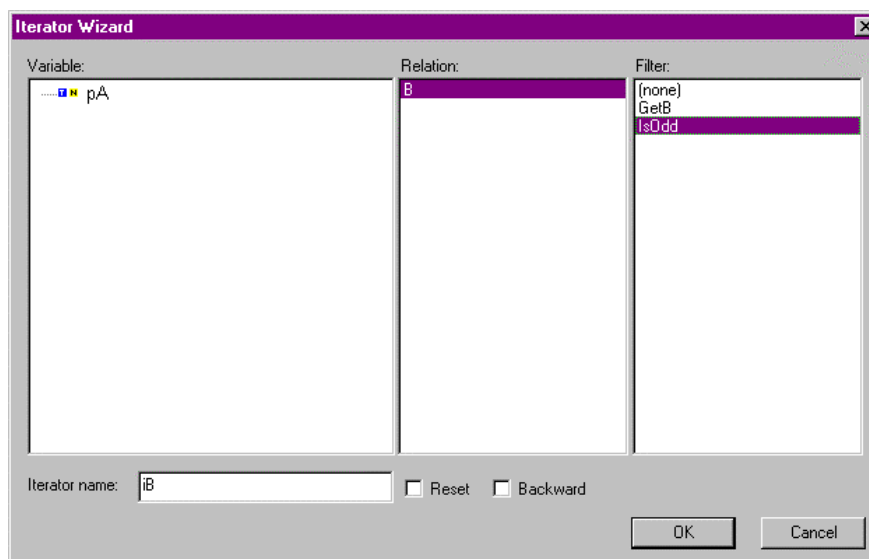
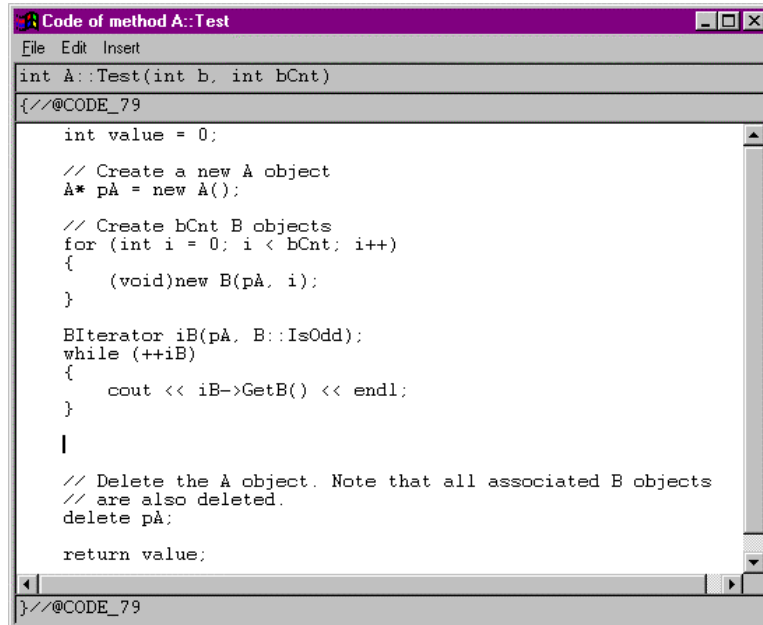


Figure 2-29: Dialog to insert iterator code.

The selected settings mean, we start an iteration at the object pointed to by 'pA'. It iterates the relation 'B' and it uses the method 'IsOdd()' as filter. This filter means

that only those objects are iterated where the 'IsOdd()' method would return a non zero value. To phrase things differently, all 'B' objects where 'IsOdd()' returns a zero value are skipped. Note that all non static methods returning an 'int' and having no arguments, can be used as filter method.

22. Add the line 'cout << iB->GetB() << endl;' in the inserted loop and position your cursor as shown in Figure 2-30.



```

Code of method A::Test
File Edit Insert
int A::Test(int b, int bCnt)
{
    //@@CODE_79
    int value = 0;
    // Create a new A object
    A* pA = new A();
    // Create bCnt B objects
    for (int i = 0; i < bCnt; i++)
    {
        (void)new B(pA, i);
    }
    BIterator iB(pA, B::IsOdd);
    while (++iB)
    {
        cout << iB->GetB() << endl;
    }
    // Delete the A object. Note that all associated B objects
    // are also deleted.
    delete pA;
    return value;
}
//@@CODE_79

```

Figure 2-30: The code of method 'A::Test()', after inserting iterator.

23. Select **Insert**→**if () {}** from the pop up menu.
24. Select **Insert**→**variable->Method()**.

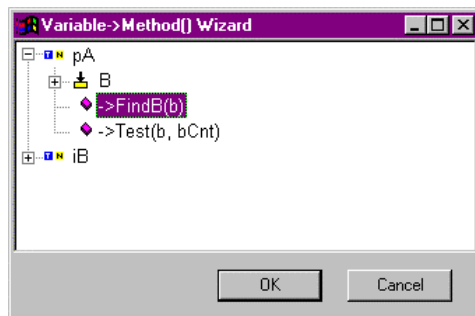


Figure 2-31: The 'variable->Method()' wizard dialog.

25. In the new popped up window, expand item **pA** by clicking on its plus sign located on its left.
26. Select item **>FindB(b)** and click on the 'OK' button, if your window looks the same as Figure 2-31.

The idea of this dialog box is that you first select a variable, then apply a navigation on this variable and finally select a method. In this manner code is inserted where a method is called with possible complex navigation, which is correct by construction.

27. Add the line 'value = 1;,' see Figure 2-32.

```

Code of method A::Test
File Edit Insert
int A::Test(int b, int bCnt)
{
    //@@CODE_79
    int value = 0;

    // Create a new A object
    A* pA = new A();

    // Create bCnt B objects
    for (int i = 0; i < bCnt; i++)
    {
        (void)new B(pA, i);
    }

    BIterator iB(pA, B::IsOdd);
    while (++iB)
    {
        cout << iB->GetB() << endl;
    }

    if (pA->FindB(b))
    {
        value = 1;
    }

    // Delete the A object. Note that all associated B objects
    // are also deleted.
    delete pA;

    return value;
}
//@@CODE_79

```

Figure 2-32: The finished code of 'A::Test()'.

27. Save the changes made to the code either by pressing **Ctrl+S** or by selecting *File*→*Save* from the code window menu.
28. Close the code window of 'A::Test()'.

2.8 Saving your design and generating source code

It is time to save all our efforts so far and to generate code.

1. Save the whole design by selecting *File*→*Save* from the main window or by pressing **Ctrl+S**, the following file dialog appears, see Figure 2-33.

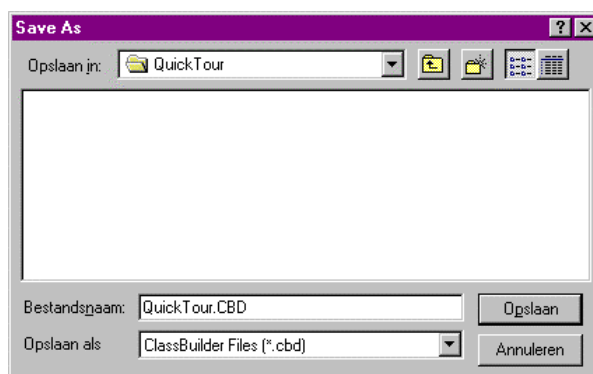


Figure 2-33: The file dialog to save your QuickTour.

2. Save your design in an appropriate directory, your source code is written in the same directory as your design file!

Now we are ready to generate C++ source files.

3. Press the  *Save Source* button on the tool bar (*File*→*Save Source*).
4. A dialog box is displayed, see Figure 2-34, give an appropriate text to the 'Author & Note' field and click on the 'Save All' button.

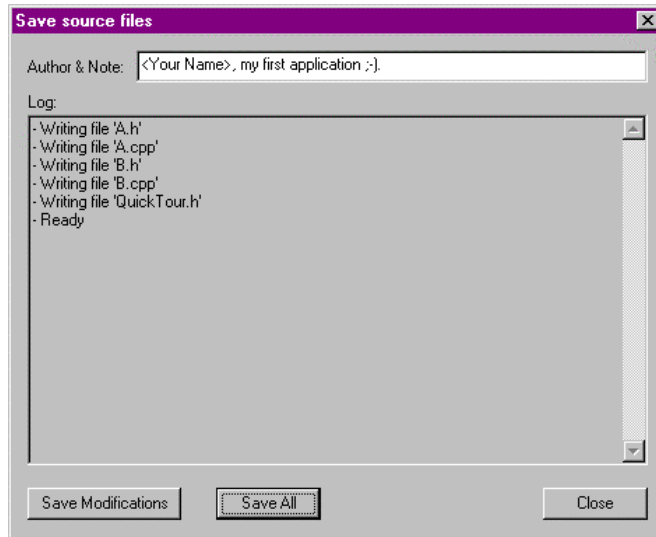


Figure 2-34: The dialog to save the generated source code.

Now the source files are written to the same directory as the design file.

5. Click on the 'Close' button to close this window.

2.9 Finishing the program

We finish the program with Visual C++ 4.2, we assume that the reader is familiar with this compiler environment. The steps you have to make to finish the program in this environment are not explained in detail.

1. Create a new console application project named QuickTour, add the generated files 'A.cpp' and 'B.cpp' to it.
2. Make sure the compiler can find the include files needed for ClassBuilder generated source code.
3. Edit the file 'QuickTour.h' and add the line '#include <iostream.h>', see figure Figure 2-35 where to put this line.

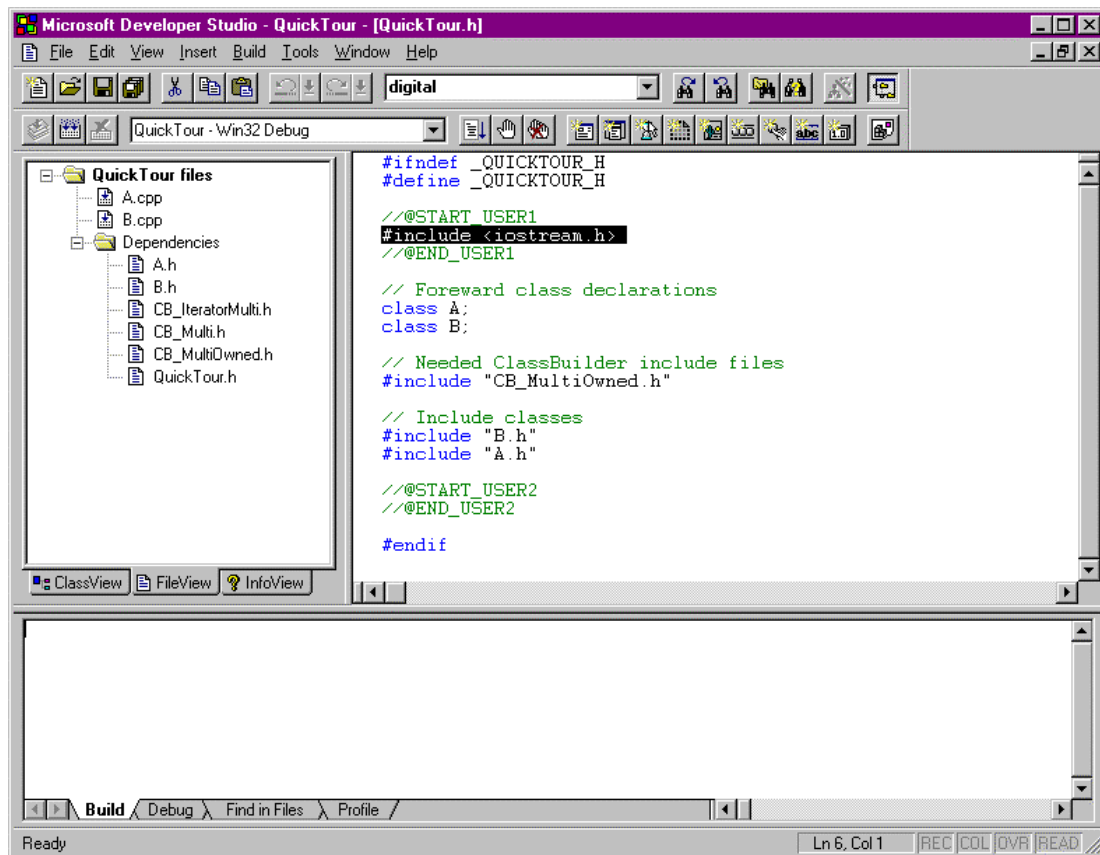


Figure 2-35: Add '#include<iostream.h>' to 'QuickTour.h'

4. Create a new file 'Main.cpp' and add this file to the project.
5. Edit the code of 'Main.cpp' as shown in Figure 2-36.

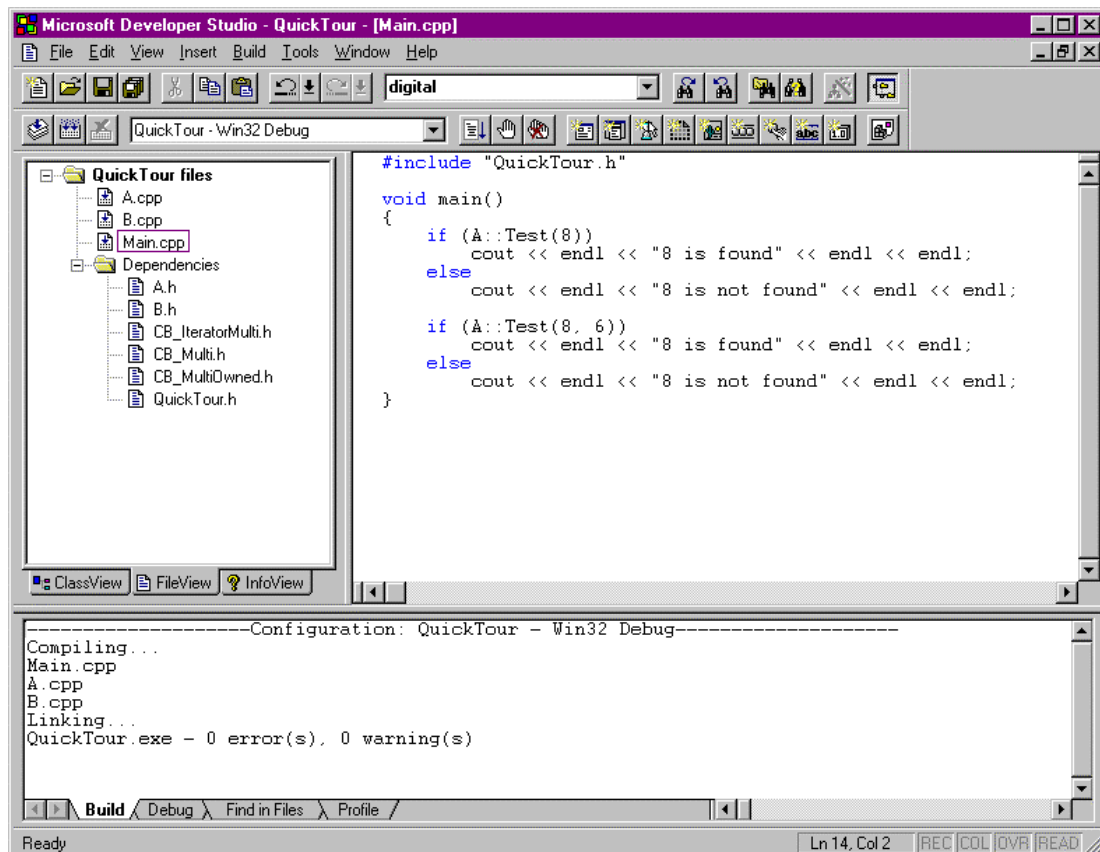


Figure 2-36: The code of 'Main.cpp'.

6. Compile the program and run it, the output generated by the program is shown in Figure 2-37.

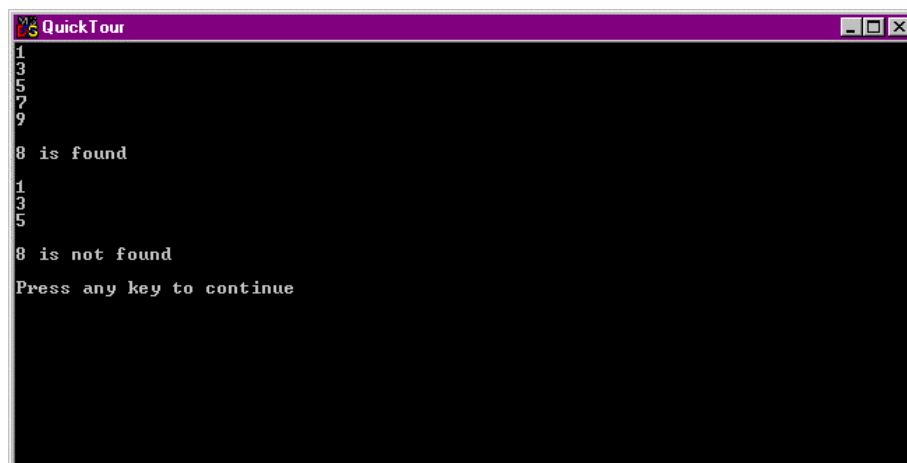


Figure 2-37: The output generated by the QuickTour program.

Now we read back the changes from the compile environment into the ClassBuilder environment.

7. Click on the Read Source button on the tool bar.
8. In the new window displayed, click on the 'Read' button the result is shown in Figure 2-38.

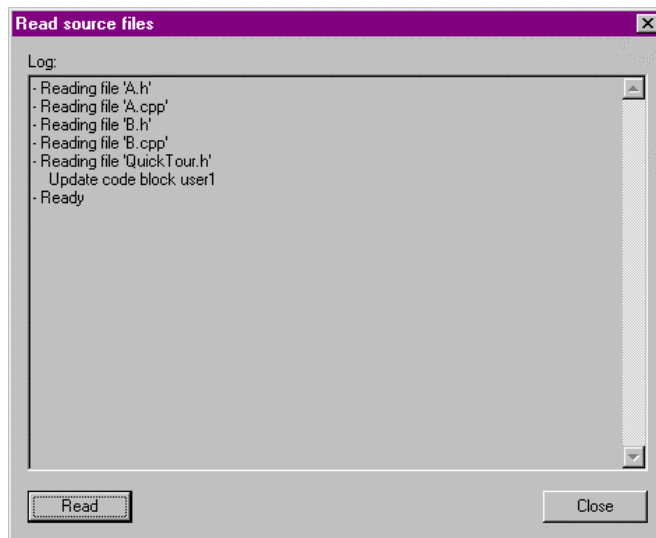


Figure 2-38: Reading back the changes into the ClassBuilder environment.

As can be seen only the user1 code block of file 'QuickTour.h' is updated. The file 'Main.cpp' isn't known in the ClassBuilder environment, so its changes are not read in.

Lets make a change to method 'Test' of class 'A' in the compiler environment.

9. Open file 'A.cpp' in Visual C++.

10. Add the comment line "*// Iterate the odd 'B' objects*" as shown in Figure 2-39.

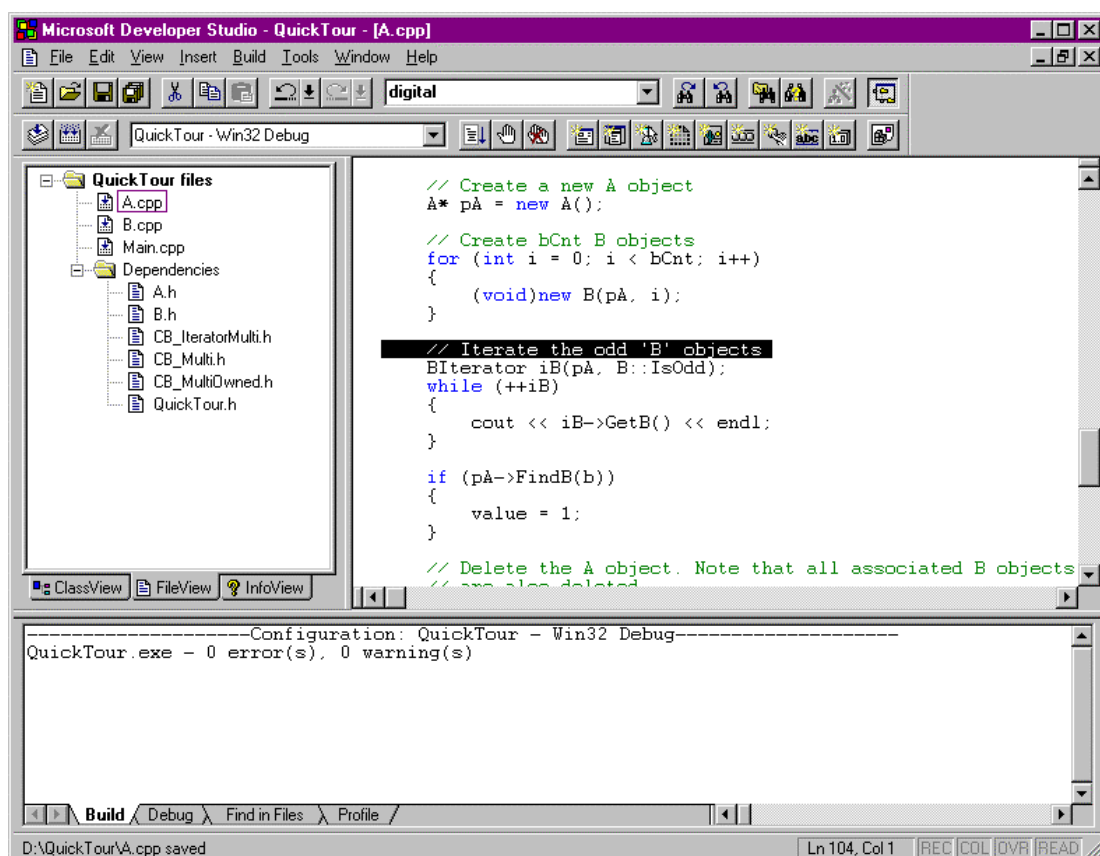


Figure 2-39: Add comment line in code of method 'A::Test()'.
 11. Save the change in the compile environment.

12. Read again the source in ClassBuilder (Step 7).

See that it has noticed an update of the 'Test' method in file 'A.cpp'.

13. Close the read source window by clicking on the 'Close' button.

14. Double click on the  static int Test(int b, int bCnt = 10) method item to open the code editor and notice the update of the code, see Figure 2-40.

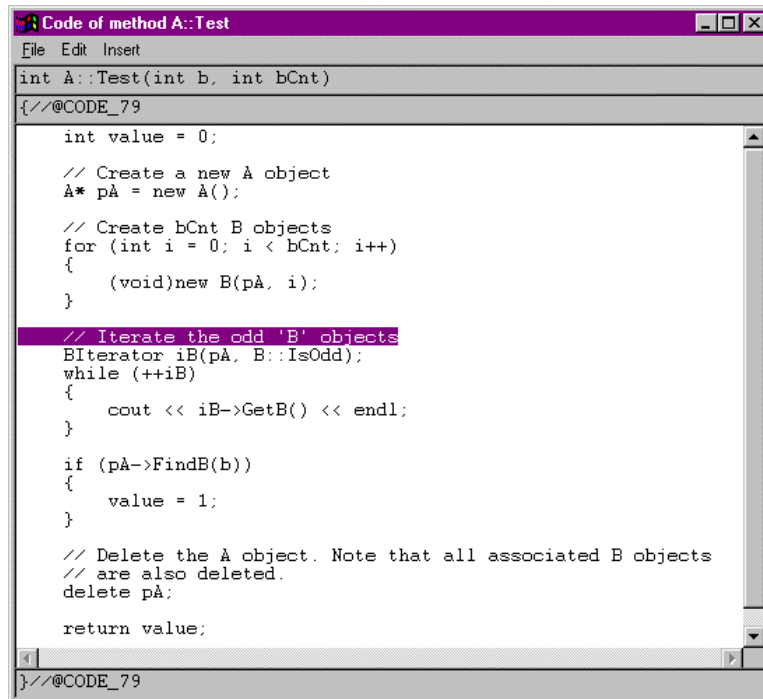


Figure 2-40: The updated code of method 'A::Test()'.

15. Close the code window.

2.10 Generating documentation

ClassBuilder can generate documentation in both RTF and HTML format. We demonstrate how the generation works for the HTML format.

1. Select from the main menu *File*→*Write Documentation (Html)*, the following dialog is displayed, see Figure 2-41.

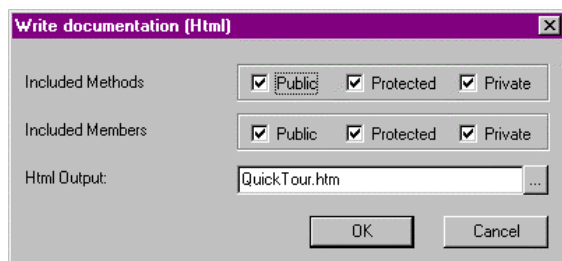


Figure 2-41: The write html documentation dialog.

2. Click on the 'Ok' button.

By default the Html files are written in the same directory as the design file. To select another directory just click on the '...' button to select another directory.

The result as displayed with a browser is shown in Figure 2-42. Via the links one can easily browse through the document.

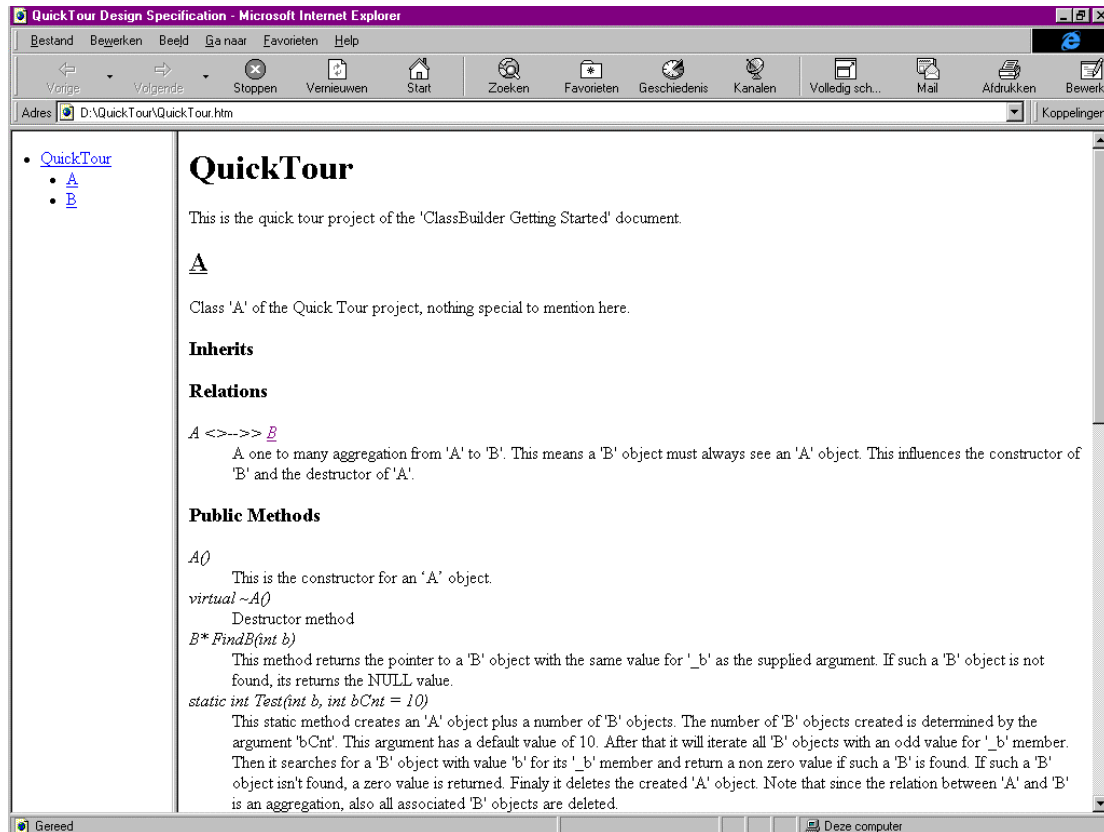


Figure 2-42: The generated Html documentation loaded in a browser.

This was our tour, we hope you enjoyed it!

3. Relations and lifetime of objects

Relations can have an effect on the lifetime of the objects they relate. It is necessary to elaborate a bit about this subject to understand the behaviour of the generated code. First we dig into the lifetime of objects and relations. Then we will discuss in detail each relation type the ClassBuilder tool supports.

3.1 Life time

3.1.1 Definitions

Lets start with some definitions of the symbols used.

$$Time = \langle -\infty, \infty \rangle$$

$$L \subset Time$$

$$L_A = \text{Lifetime of an object of class } A$$

$$L_R = \text{Lifetime of a relation between two objects}$$

3.1.2 Association

Let us consider the situation of an association between an object of class 'A' and an object of class 'B'. The multiplicity of the association doesn't matter.



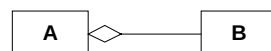
The lifetimes for an association between 'A' and 'B' are:

$$L_R \subset L_A \wedge L_R \subset L_B \Rightarrow L_R \subset L_A \cap L_B$$

Or to state in words, the relation between an object of class 'A' and an object of class 'B' can only exists if both objects exists. So if an objects stops to exists, all associations it is part of also stops to exists. A good place to force this behaviour into the objects would be the destructor in case of using C++.

3.1.3 Aggregation

Now we will consider the situation of an aggregation between an object of class 'A' and an object of class 'B'. It doesn't matter if it is an one to one or an one to many aggregation. Sometimes in drawings we see the association notation, plus the figure '1' at one end. Since this has the same lifetime definition as an aggregation, we tread it as an aggregation.

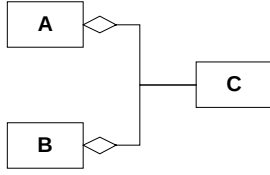


The lifetimes for an aggregation between 'A' and 'B' are:

$$L_R \subset L_A \wedge L_R = L_B \Rightarrow L_B \subset L_A$$

The difference with the above association is that the lifetime of the relation is now equal to the life time of the 'B' object. So if a 'B' object exists, also a relation must exist to relate it to an 'A' object. And if an 'A' object stops to exists, all 'B' objects part of its aggregation stops to exist. Good places to force this behaviour into the objects are in the constructor and destructor.

The situation of two objects (A, B), both aggregating the same object (C), is just a natural extension of one object and its aggregate.



$$L_{RAC} = L_{RBC} = L_R$$

$$L_R \subset L_A \cap L_B \wedge L_R = L_C \Rightarrow L_C \subset L_A \cap L_B$$

Or to put it in words, objects of class ‘C’ can only exist if they are related to an ‘A’ object and a ‘B’ object. It is important to understand that if the above class diagram is specified that the pre and post condition of every public method operating on these classes is the fulfillment of the specification. A situation of an in core data structure which isn’t according to specification should never occur. This is the reason why the correct lifetime behaviour is forced into the constructor and destructor by the ClassBuilder tool.

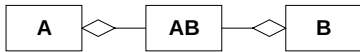
3.1.4 Transformations

With the definitions of lifetime one can easily see that transforming an association into two aggregations is a valid transformation. This is important, since ClassBuilder doesn’t support the direct implementation of a many to many association. Lets first look again at the definition for an association between two classes.



$$L_R \subset L_A \cap L_B$$

Next compare it with the definitions of modeling it with 3 classes and 2 aggregations.



$$L_{AB} \subset L_A \cap L_B$$

As we can see, the transformation of the modeling seems a valid one. So a many to many association can be modeled with an extra class and two one to many aggregations.

3.1.5 The effect on constructors and destructors

As mentioned above a good place to force the correct lifetime behaviour into the objects are the constructor and destructor. This is exactly what ClassBuilder does, the behaviour as implemented by ClassBuilder is listed in Table 1.

	A ---- B	A <>--- B
Constructor A	-	-
Constructor B	-	Add to aggregation.
Destructor A	Remove from association.	Destroy aggregates.
Destructor B	Remove from association.	Remove from aggregation.

Table 1: Behaviour of constructors and destructors.

ClassBuilder implements this behaviour with the use of the 'ConstructorInclude()' and the 'DestructorInclude()' methods. The 'ConstructorInclude()' method is responsible for the correct lifetime behaviour when constructing objects and the 'DestructorInclude()' method when destructing objects. These methods must always be invoked in the constructor and destructor. When ClassBuilder generates constructors and destructors, it generates these invocations also, don't remove them from the code!

3.2 Single associations

A single association relates an **object** of a class to another **object** of the same or other class. One object is the active object, the other object is the passive object. The active object is in control of the relation. The methods for the creation and deletion of the relation are declared and implemented at the class of the active object. These methods are applied to the active object to actually control the relation of the active object. To the passive object only one method can be applied, and this method returns the active object. The deletion of the passive object is reported to the active object and the relation between the two is deleted. The DestructorInclude() method is responsible for this behaviour, so it is important to call this method in the destructor! The generated template code for a destructor does include this call.

The relation dialog should have the settings as showed in Figure 3-1 to have a single association between an object of class 'A' and an object of class 'B'. The 'From' class 'A' is the active side of the association, 'To' class 'B' the passive side. Figure 3-2 gives an overview of the generated methods.

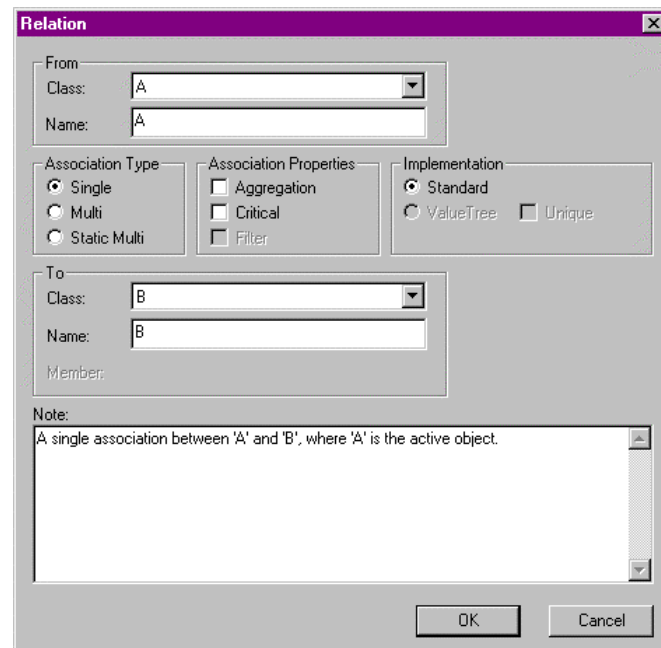


Figure 3-1: Single association between 'A' and 'B'.

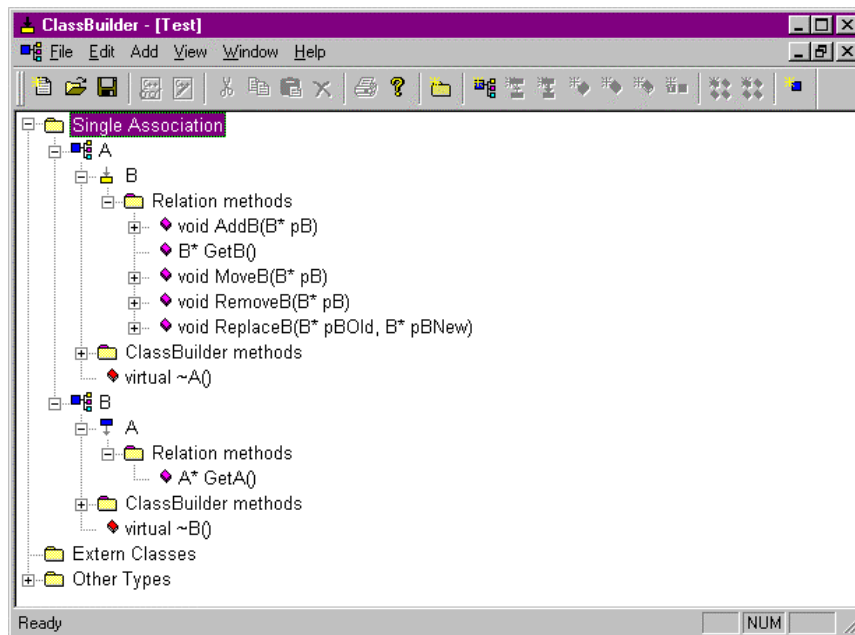


Figure 3-2: The generated methods for a single association.

3.2.1 Active object methods

Below the generated methods of the active class 'A' are described in more detail. These methods are not directly visible in the generated code, but are inserted via a macro.

3.2.1.1 Relation addition method

void AddB(B* pB);

Add the B object where 'pB' points to as passive object to the relation of the current active object. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is the relation of the current object empty? (this is a single relation!)
- Is 'pB' a valid object?
- Is 'pB' not already part of a relation of this type?

3.2.1.2 Relation remove method

void RemoveB(B* pB);

From the relation of the current active object, remove the passive B object pointed to by 'pB'. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?
- Is 'pB' the passive object of the current object relation?

3.2.1.3 Relation move method

void MoveB(B* pB);

Move the B object pointed to by 'pB', to the relation of the current active object. If 'pB' is related to another active object, it is first removed from this relation. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is the relation of the current object empty? (this is a single relation!)

- Is 'pB' a valid object?

3.2.1.4 Relation substitution method

void ReplaceB(B* pB, B* pBNew);

From the relation of the current active object, replace the passive B object pointed to by 'pB' with the passive B object pointed to by 'pBNew'. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?
- Is 'pBNew' a valid object?
- Is 'pB' the passive object of the current object relation?
- Is 'pBNew' not already part of a relation of this type?

3.2.1.5 Relation interrogation method

B* GetB();

Return the passive object of the current object relation. If the relation is empty '(B*)0' is returned. With assert statements the following precondition checks are performed:

- Is the current object a valid object?

3.2.2 Passive object methods

Below the generated method for class 'B' is described in more detail.

3.2.2.1 Relation interrogation method

A* GetA();

Return the active object of the current object relation. If the passive object isn't related to an active object '(A*)0' is returned. With assert statements the following precondition checks are performed:

- Is the current object a valid object?

3.3 Single Aggregations

The difference with the single association is that the active object now literally owns the passive object. No object of the passive object class may exist without being part of this relation. This behaviour is forced with the ConstructorInclude() method. The ConstructorInclude() method of the passive object class has as argument the pointer to the active object. It is obligatory to call the ConstructorInclude() method first in a non replace constructor! This way it is impossible to construct a passive object without relating it to its active object.

The same way as the constructor of the passive object creates the relation, the destructor of the passive object deletes the relation. The DestructorInclude() method takes care of this behaviour. The destructor of the active object, also deletes its owned passive object, no passive owned object may exist without its active object.

The relation dialog should have the settings as showed in Figure 3-3 to have a single aggregation between an object of class 'A' and an object of class 'B'. Figure 3-4 gives an overview of the generated methods.

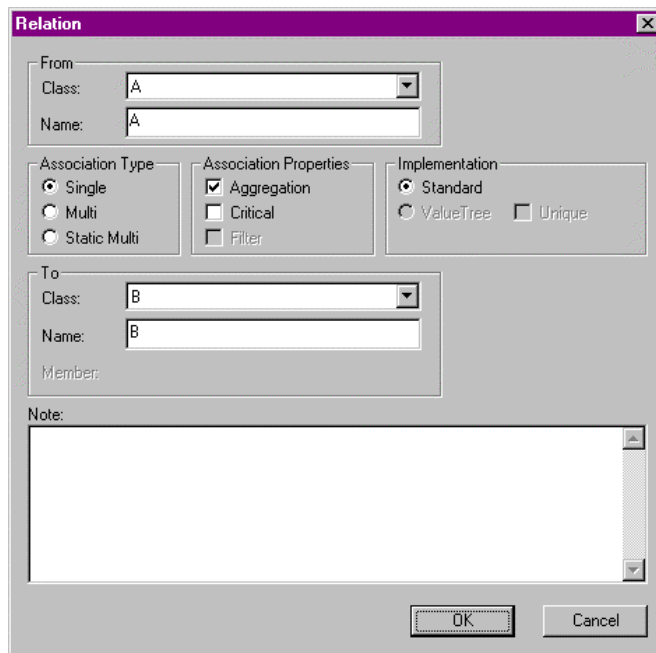


Figure 3-3: Single aggregation between 'A' and 'B'.

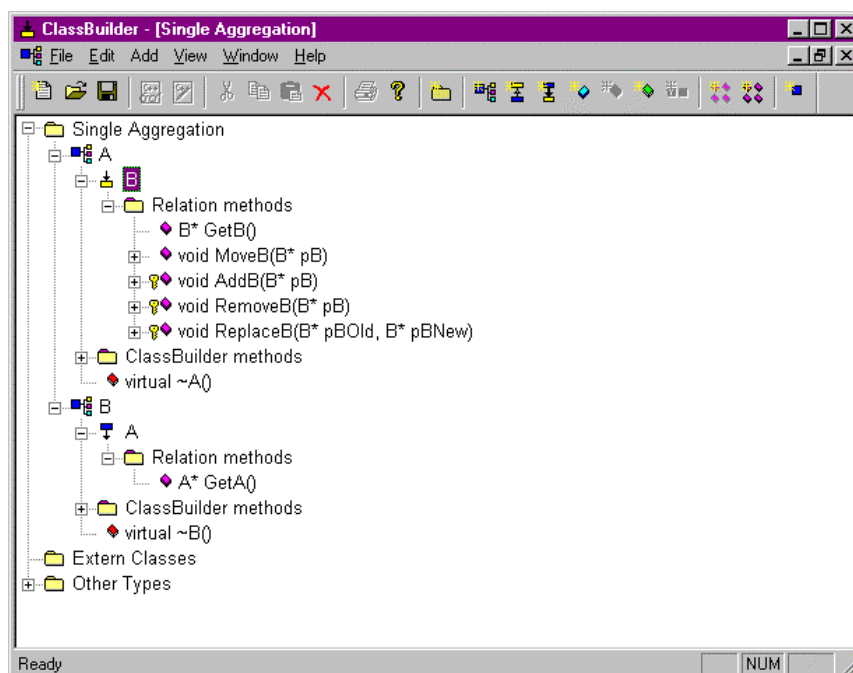


Figure 3-4: The generated methods for a single aggregation.

The single aggregation has the same methods as the single association. The difference is that the 'Relation addition method', 'Relation remove method' and 'Relation substitution method' are now protected methods instead of public. Direct creation or deletion of a relation is prohibited this way.

3.4 Naming single relations

If more than one relation exist between two classes, name clashes in member and method names can occur. To circumvent this, a relation can be named. Figure 3-5 shows an example of a named relation.

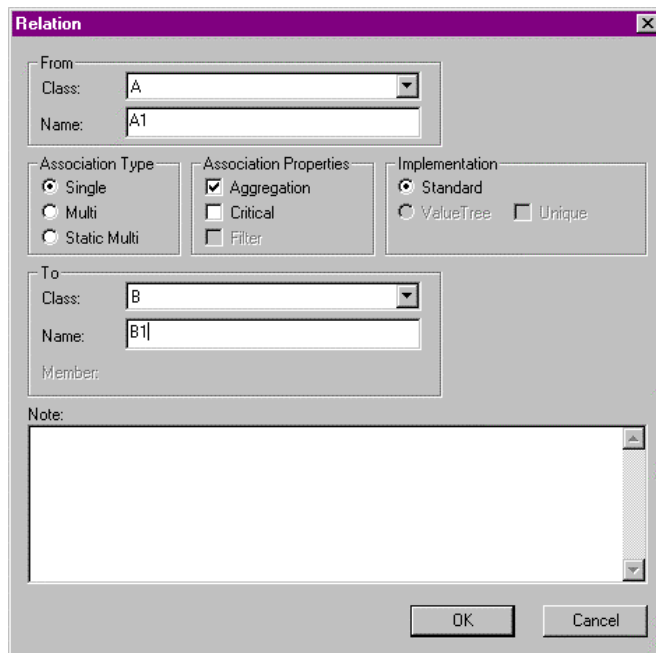


Figure 3-5: Naming a relation.

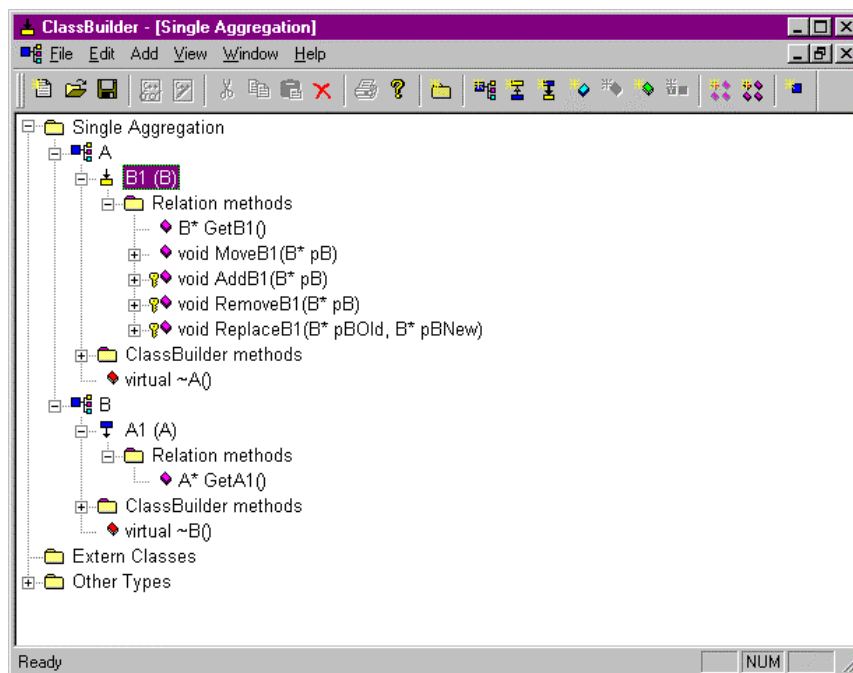


Figure 3-6: The methods, if a relation is named.

Figure 3-6 shows the generated methods, when the relation is named as in Figure 3-5.

3.5 Multi associations

A multi association relates an **object** of a class to other **objects** of a class. The object is the active object, while the other objects it relates to are the passive objects. The class of the active object may differ from the class of the passive objects. The active object is in control of the relation. The methods for the addition and the removal of passive objects to the relation are declared and implemented at the class of the active object. These methods are applied to the active object to actually control the relation

of the active object. To a passive object only one method can be applied, this method returns the active object. The deletion of a passive object is reported to its active object and the passive object is removed from the association. The `DestructorInclude()` method is responsible for this behaviour, so it is important to call this method in the destructor! The generated template for a destructor does include this call.

The relation dialog should have the settings as showed in Figure 3-7 to have a multi association between an object of class 'A' and an object of class 'B'. The 'From' class 'A' is the active side of the association, 'To' class 'B' the passive side. Figure 3-8 gives an overview of the generated methods.

The screenshot shows a 'Relation' dialog box with the following settings:

- From:**
 - Class: A
 - Name: A
- Association Type:**
 - ☐ Single
 - ☒ Multi
 - ☐ Static Multi
- Association Properties:**
 - ☐ Aggregation
 - ☐ Critical
 - ☒ Filter
- Implementation:**
 - ☒ Standard
 - ☐ ValueTree
 - ☐ Unique
- To:**
 - Class: B
 - Name: B
 - Member: (empty)
- Note:** (empty text area)
- Buttons:** OK, Cancel

Figure 3-7: Multi association between 'A' and 'B'.

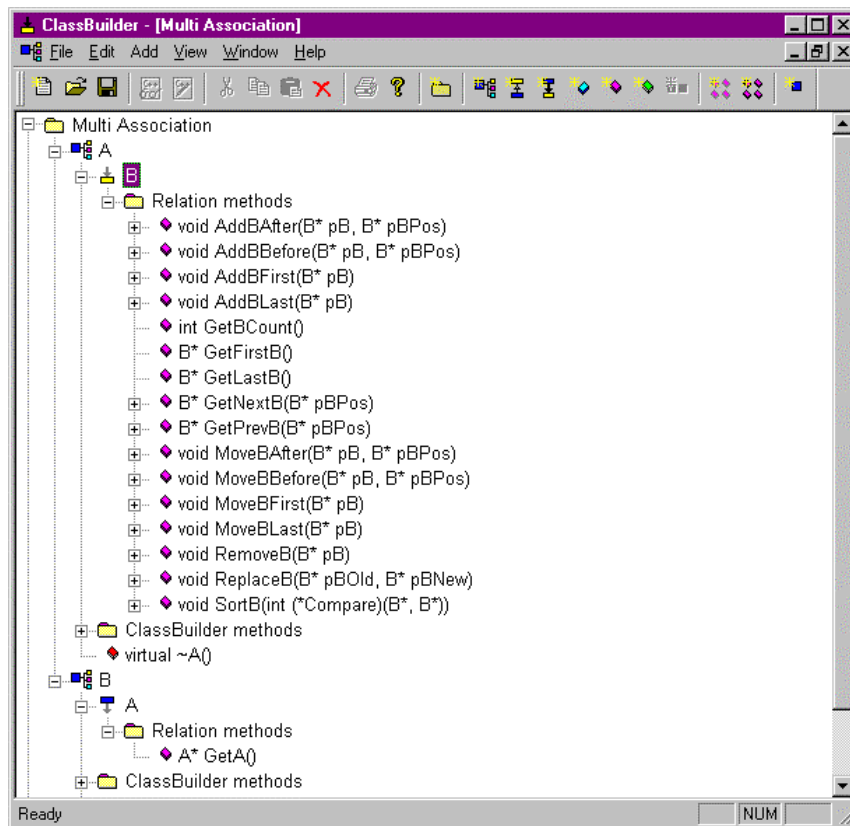


Figure 3-8: The generated methods for a multi association.

3.5.1 Active object methods

Below, the methods for class 'A' are described in more detail, note that these methods aren't directly visible in the generated code, they are inserted with a macro.

3.5.1.1 Relation addition methods

void AddBFirst(B* pB);

Add the B object where 'pB' points to as first passive object to the relation of the current active object. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?
- Is 'pB' not already part of a relation of this type?

void AddBLast(B* pB);

Add the B object where 'pB' points to as last passive object to the relation of the current active object. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?
- Is 'pB' not already part of a relation of this type?

void AddBAfter(B* pB, B* pBPos);

Add after the B object pointed to by 'pBPos', the B object pointed to by 'pB', as a passive object to the relation of the current active object. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?
- Is 'pB' not already part of a relation of this type?
- Is 'pBPos' a valid object?
- Is 'pBPos' a passive object of the current object relation?

void AddBBefore(B* pB, B* pBPos);

Add before the B object pointed to by 'pBPos', the B object pointed to by 'pB', as a passive object to the relation of the current active object. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?
- Is 'pB' not already part of a relation of this type?
- Is 'pBPos' a valid object?
- Is 'pBPos' a passive object of the current object relation?

3.5.1.2 Relation remove method

void RemoveB(B* pB);

Remove the passive B object where 'pB' points to from the relation of the current active object. With assert statements the following precondition checks are performed:

- Is the current object a valid object.
- Is 'pB' a valid object.
- Is 'pB' a passive object of the current object relation.

3.5.1.3 Relation substitution method

void ReplaceB(B* pBOld, B* pBNew);

Remove the passive B object where 'pBOld' points to from the relation of the current active object and add 'pBNew' in 'pBOld' place. With assert statements the following precondition checks are performed:

- Is the current object a valid object.
- Is 'pBOld' a valid object.
- Is 'pBNew' a valid object.
- Is 'pBOld' the passive object of the current object relation.
- Is 'pBNew' not already part of a relation of this type.

3.5.1.4 Relation interrogation methods

B* GetFirstB();

Return the first passive B object of the current object relation, if the current object does not relate to any B object '(B*)0' is returned. With assert statements the following precondition checks are performed:

- Is the current object a valid object.

B* GetLastB();

Return the last passive B object of the current object relation, if the current object does not relate to any B object '(B*)0' is returned. With assert statements the following precondition checks are performed:

- Is the current object a valid object?

B* GetNextB(B* pBPos);

Return the next passive B object of the current object relation relatively to the 'pBPos' object. If the 'pBPos' object is the last object of the current object relation, then '(B*)0' is returned. If 'pBPos' is the NULL pointer the first passive B object of the current object relation is returned. It is impossible to ask the next of a deleted object! With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pBPos' a valid object?
- Is 'pBPos' a passive object of the current object relation?

B* GetPrevB(B* pBPos);

Return the previous passive B object of the current object relation relatively to the 'pBPos' object. If the 'pBPos' object is the first object of the current object relation, then '(B*)0' is returned. If pBPos is the NULL pointer the last passive B object of the current object relation is returned. It is impossible to ask the previous of a deleted object! With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pBPos' a valid object?
- Is 'pBPos' a passive object of the current object relation?

int GetBCount();

Return the number of passive objects of the current object relation. With assert statements the following precondition check is performed:

- Is the current object a valid object?

3.5.1.5 Relation reorder methods

void MoveBFirst(B* pB);

Move the B object pointed to by 'pB', as first passive object to the relation of the current active object. If 'pB' is related to another active object, it is first removed from this relations. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?

void MoveBLast(B* pB);

Move the B object pointed to by 'pB', as last passive object to the relation of the current active object. If 'pB' is related to another active object, it is first removed from this relations. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?

void MoveBAfter(B* pB, B* pBPos);

Move the B object where 'pB' points to after the 'pBPos' object. If 'pB' is related to another active object, it is first removed from this relations. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?
- Is 'pBPos' a valid object?

- Is ‘pBPos’ a passive object of the current object relation?
- Are ‘pB’ and ‘pBPos’ not the same objects?

```
void MoveBBefore(B* pB, B* pBPos);
```

Move the B object where ‘pB’ points to before the ‘pBPos’ object. If ‘pB’ is related to another active object, it is first removed from this relations. With assert statements the following precondition checks are performed:

- Is the current object a valid object?
- Is 'pB' a valid object?
- Is 'pBPos' a valid object?
- Is 'pBPos' a passive object of the current object relation?
- Are 'pB' and 'pBPos' not the same objects?

```
void SortB(int (*comp)(B*, B*));
```

Sort the relation of the current object while using ‘comp’ as the compare function. The compare function must return a negative number if its first argument is of lower order than its second argument. If the first argument is of higher order than the second argument, then the compare function must return a positive number. In case the two arguments are of the same order the compare function should return zero. With assert statements the following precondition checks are performed:

- Is the current object a valid object?

An example of the use of the Sort() method. Class A has a multi relation with class B and class B has an extra int data member ‘_value’ with retrieve method ‘int GetValue()’.

```
int Compare(B* a, B* b)
{
    return a->GetValue() - b->GetValue();
}

void SomeFunction(..)
{
    ..
    A* a = ..;
    ..
    a->SortB(Compare);
}
```

3.5.2 Iterator class

For each multi association an Iterator class is declared and implemented. They simplify the iteration of the relation. They are declared within the active class. If they are used outside the context of the active class, then they must be referenced with the active class name first followed by two double colons. An example of the use of the Iterator class. Class A has a multi relation with class B and class B has an extra int data member ‘_value’ with retrieve method ‘int GetValue()’. Suppose we want to print the value of the B objects an A object relates to.

```
void PrintValues(A* a)
{
    A::BIterator b(a);
    while (++b)
        cout << b->GetValue() << endl;
}
```

Note that we use `b` as if it was a pointer to `B`. Within the `A::BIterator` class the cast ‘`operator B*()`’ and the ‘`operator->()`’ is defined to return the `B` pointer where the Iterator is currently pointing at. It isn’t necessary to precede ‘`BIterator`’ with ‘`A::`’ if

‘PrintValues()’ is a method of the active class ‘A’. Below the same example as above, but now ‘PrintValues()’ is a method of ‘A’.

```
void A::PrintValues()
{
    BIterator b(this);
    while (++b)
        cout << b->GetValue() << endl;
}
```

It is safe to delete, remove or replace objects while iterating them with the Iterator class. When deleting, removing or replacing an object, it is checked if that object is referenced by any active Iterator, if such situation occurs, that Iterator is updated! First an example of this, without using the Iterator class. Class A has a multi relation with class B and class B has an extra int data member ‘_value’ with retrieve method ‘int GetValue()’. Suppose we want to delete all B objects with an odd value. First an erroneous example:

```
// Faulty example
void A::DeleteOddValues()
{
    for (B* b = GetFirstB(); b; b = GetNextB(b))
    {
        if (b->GetValue() & 0x1)
            delete b; // Error
    }
}
```

We can not delete b directly, since we still want to use it in the GetNextB() in the for statement. If we go back to section 3.5.1.4 we see that one of the preconditions for GetNextB() is a valid argument. A pointer to a deleted object, isn’t exactly a valid argument. This error can be avoided, an example of it is shown below.

```
// Correct example
void A::DeleteOddValues()
{
    for (B* b = GetFirstB(); b; b = GetNextB(b))
    {
        if (b->GetValue() & 0x1)
        {
            b = GetPrevB(b); // Set b at previous, so that
                            // GetNextB() in for statement works
            delete GetNextB(b); // Delete the correct b
        }
    }
}
```

The code is a little tricky, the use of the Iterator class gives cleaner code, below the same example, but with the Iterator class used.

```
void A::DeleteOddValues()
{
    BIterator b(this);
    while (--b)
        if (b->GetValue() & 0x1)
            delete (B*)b; // or delete b.Get();
}
```

For the delete statement the automatic cast won’t work, so we help the compiler. Note that we can also iterate backwards with the ‘—’ operator.

An overview of all methods of the Iterator class.

```
BIterator(A* iterA, int (B::*method)() = 0, B* refB = (B*)0);
BIterator(A& iterA, int (B::*method)() = 0, B* refB = (B*)0);
BIterator(const BIterator& iterator, int (B::*method)() = 0);
BIterator& operator= (const BIterator& iterator);
B* operator++ ();
B* operator-- ();
void Reset();
operator B*();
B* operator-> ();
```

```

    B* Get();
    int IsFirst();
    int IsLast();

```

Below, the methods are described in more detail, note that these methods aren't directly visible in the class definition, they are inserted with a macro defined in one of the include files.

3.5.2.1 Construction methods

```

    BIterator(A& iterA, int (B::*method)() = 0, B* refB = (B*)0);
    BIterator(A* iterA, int (B::*method)() = 0, B* refB = (B*)0);

```

Construct an iterator to iterate the B objects of A object 'iterA'. With optional default argument 'refB' the iteration can start somewhere else then the beginning or the end. With the default argument 'method' a pointer to a method of class 'B' can be supplied. This method acts as a filter for the iteration. Only pBs are iterated where the supplied method returns a non zero value. The previous example, to delete all 'B' objects with an odd value, can be rewritten if there is a method 'int B::IsOdd()':

```

void A::DeleteOddValues()
{
    BIterator b(this, B::IsOdd);
    while (--b)
        delete b.Get();
}

```

If the NULL value or no value at all is passed as argument to the 'method' argument, then no filtering is applied.

```

    BIterator(const BIterator& iterator, int (B::*method)() = 0);

```

Construct an iterator to iterate B objects, make it an copy of an existing Iterator. This is handy to implement a double loop on the same relation, while all combinations are unique. An example:

```

A::BIterator b1(a);
while (++b1)
{
    A::BIterator b2(b1);
    while(++b2)
        ...
}

```

With the default argument 'method' a pointer to a method of class 'B' can be supplied. This method acts as a filter for the iteration. Only 'B' objects are iterated where the supplied method returns a non zero value. If the NULL value or no value at all is passed as argument to the 'method' argument, then no filtering is applied.

3.5.2.2 Assignment method

```

    BIterator& operator= (const BIterator& iterator);

```

Make the Iterator object the same as 'iterator'

3.5.2.3 Iteration methods

```

    B* operator++ ();

```

Iterate the relation, go to the next B object and return it. (The next of the last B object is the NULL pointer.) If the method argument is used during construction of the Iterator, then the B objects where this method returns a zero value are skipped.

```

    B* operator-- ();

```

Iterate the relation, go to the previous B object and return it. (The previous of the first B object is the NULL pointer.) If the method argument is used during construction of the Iterator, then the B objects where this method returns a zero value are skipped.

void Reset();

Reset the Iterator object, with the ‘++’ operator it starts with the first B object and with the ‘—’ operator it starts at the last B object.

3.5.2.4 Cast methods

operator B*();

Return a pointer to the B object the Iterator object is referencing.

B* operator-> ();

Return a pointer to the B object the Iterator object is referencing.

B* Get();

Return a pointer to the B object the Iterator object is referencing.

3.5.2.5 Position methods

int IsFirst();

Return non zero if at first object of the iteration. Note that the filter methods are NOT taken into account.

int IsLast();

Return non zero if at last object of the iteration. Note that the filter methods are NOT taken into account.

3.5.3 Passive object methods

Below, the method for class ‘B’ is described in more detail, note that this method isn’t visible in the generated code, it is inserted with a macro.

3.5.3.1 Relation interrogation method

A* GetA();

Return the active object of the current object relation. If the passive object isn’t related to an active object ‘(A*)0’ is returned. With assert statements the following precondition checks are performed:

- Is the current object a valid object.

3.6 Multi aggregations

The active object in a multi aggregation, now literally owns the passive objects. No object of the passive objects class may exist without being part of this aggregation. This behaviour is forced with the ConstructorInclude() method of the passive objects class. This method has as argument the pointer to the active object. It is obligatory to call the ConstructorInclude() method first in a non replace constructor! This way it is impossible to construct a passive object without relating it to its active object.

The same way as the constructor of the passive object creates the relation, the destructor of the passive object deletes the aggregation. The DestructorInclude() method takes care of this behaviour. The destructor of the active object, also deletes all its owned passive objects, no passive owned object may exist without its active object.

The owned multi relation has the same relation methods as the normal multi relation. The difference is that the ‘Relation addition methods’, ‘Relation remove method’ and ‘Relation substitution method’ are now protected methods instead of public. Direct addition to a relation or removal from a relation is prohibited this way. Only the

'Relation interrogation methods' and 'Relation reorder methods' are public and thus accessible directly.

The relation dialog should have the settings as showed in Figure 3-9 to have a multi aggregation between an object of class 'A' and an object of class 'B'. Figure 3-10 gives an overview of the generated methods.

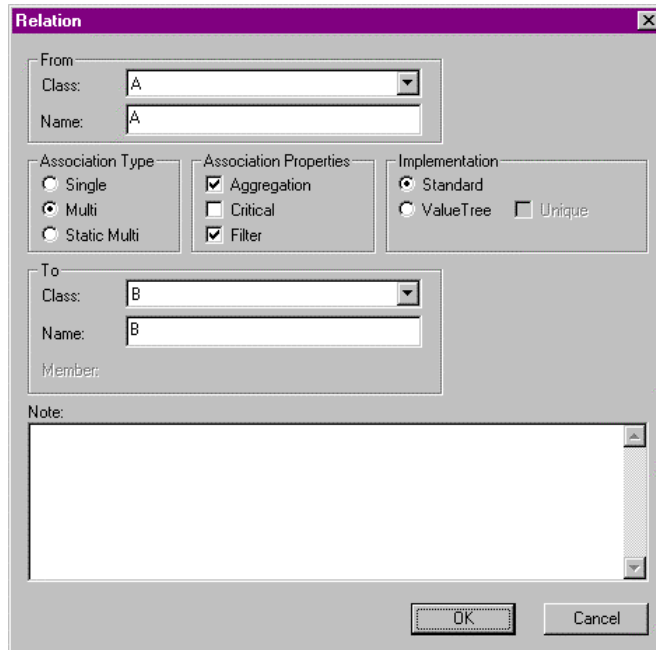


Figure 3-9: Multi aggregation between 'A' and 'B'.

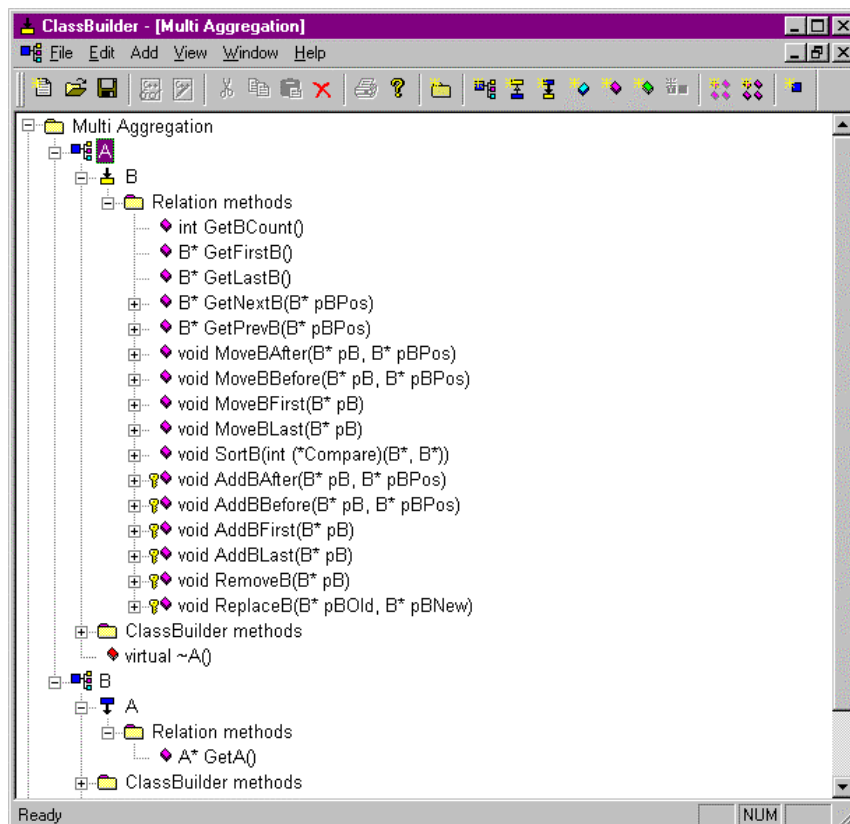


Figure 3-10: The generated methods for a multi aggregation.

3.7 Naming multi relations

If more than one relation exist between two classes name clashes in member and methods names can occur. To circumvent this, a relation can be named. Figure 3-11 shows how to set different names on a multi relation and Figure 3-12 shows the results on the method names.

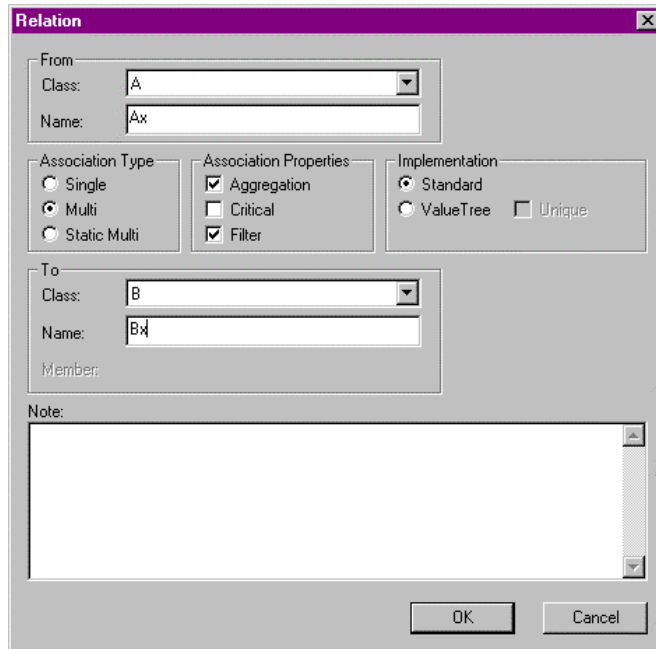


Figure 3-11: Naming a multi relation.

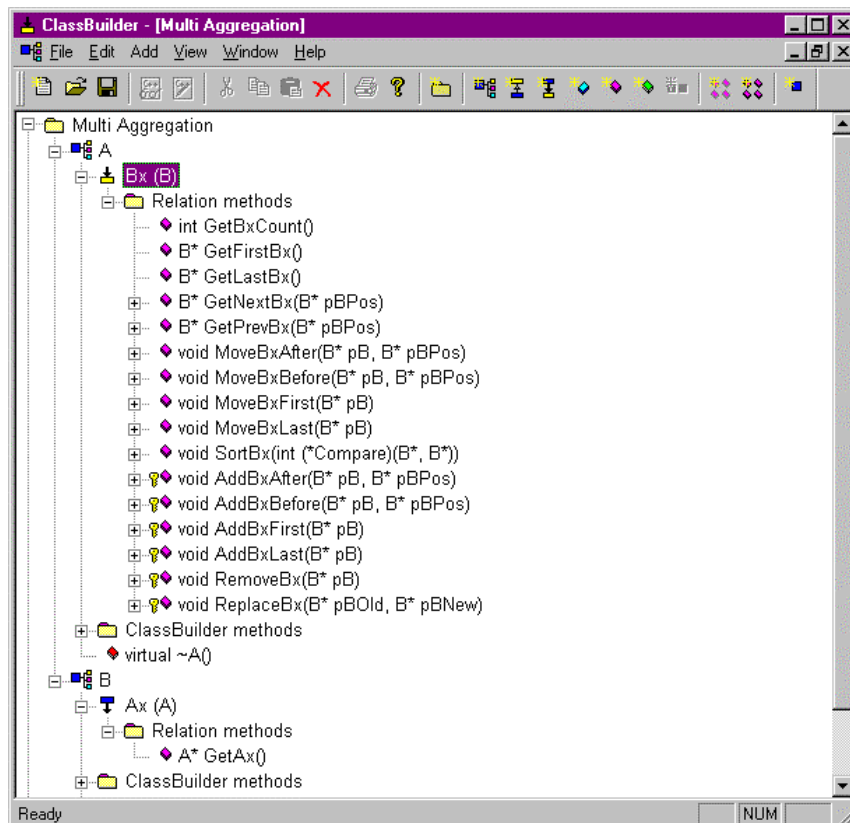


Figure 3-12: The method names of a named multi relation.

3.8 Static multi associations

A static multi relation relates a **class** to **objects** of a class. The class is the active side of the relation, the objects are passive objects. The active class may differ from the class of the passive objects. The active class is in control of the relation. The methods for the addition and the deletion of relations are declared and implemented at the active class. These methods are static by their nature, since they don't act on an object of a class. Therefore this relation is called 'static multi relation'. The deletion of a passive object is reported to the active class and the passive object is removed from the relation. The DestructorInclude() method is responsible for this behaviour, so it is important to call this method in the destructor! The generated template for a destructor does include this call.

The relation dialog should have the settings as showed in Figure 3-13 to have a static multi association between class 'A' and an object of class 'B'. Figure 3-14 gives an overview of the generated methods.

The screenshot shows a 'Relation' dialog box with the following settings:

- From:** Class: A, Name: A
- Association Type:** ☒ Static Multi
- Association Properties:** ☒ Aggregation, ☐ Critical, ☒ Filter
- Implementation:** ☒ Standard, ☐ ValueTree, ☐ Unique
- To:** Class: B, Name: B
- Note:** (Empty text area)

Buttons: OK, Cancel

Figure 3-13: Static multi association between 'A' and 'B'.

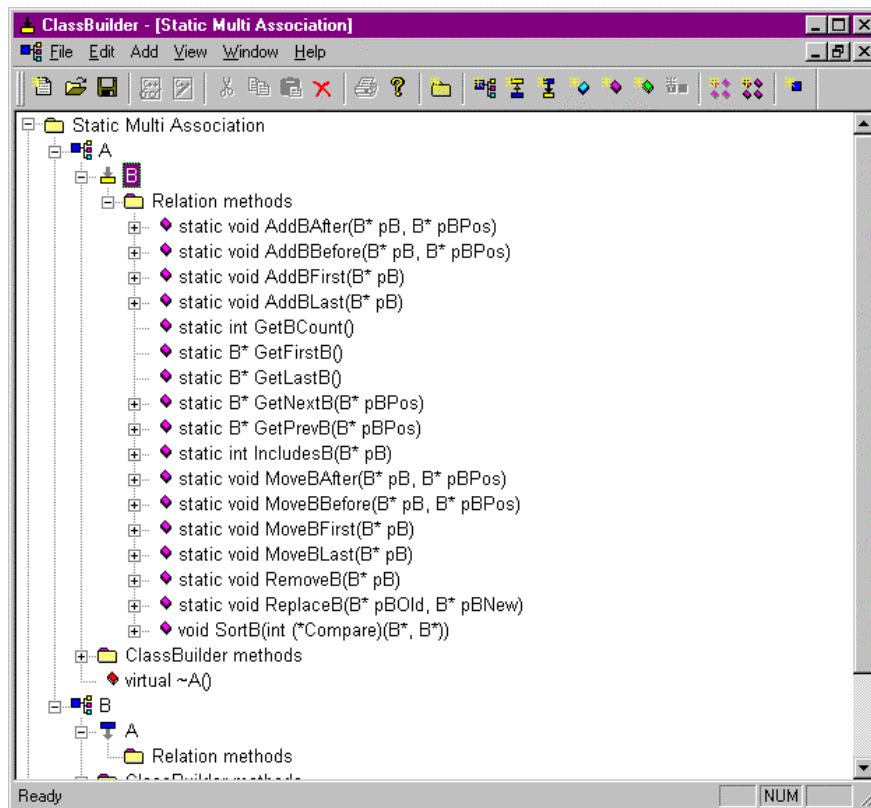


Figure 3-14: The generated methods for a static multi association.

Note that although the example above the static multi association relates the class 'A' to objects of class 'B' in practice it is more often used to relate a class to objects of the same class.

3.8.1 Active object methods

Apart from the fact that the methods are static and are class related instead of object, they act the same as for the multi association, so for more information see section 3.5.1. Same hold for the Iterator class, apart from the

BIterator(int (B::*method)() = 0, A* refA = (A*)0)

constructor the rest of the methods is the same as for a multi relation, so see section 3.5.2 for more information. The above mentioned constructor differs, since the static multi relation acts on an active class instead of an active object. There is no need for a pointer or reference to the active object.

3.8.2 Passive object methods

No methods are generated for class B (the passive object class for the static multi association from A to B).

3.9 Static multi aggregations

The class in a static multi aggregation, now literally owns the passive objects. No object of the passive objects class may exist without being part of this relation, as passive object it is always related to its active class. This behaviour is forced with the ConstructorInclude() method. The ConstructorInclude() method adds the passive object to the relation of the active class. It is obligatory to call the

ConstructorInclude() method first in a non replace constructor! This way it is impossible to construct a passive object without relating it to its active class.

The same way as the constructor of the passive object creates the relation, the destructor of the passive object deletes the relation. The DestructorInclude() method takes care of this behaviour.

The relation dialog should have the settings as showed in Figure 3-15 to have a static multi aggregation between class 'A' and an object of class 'B'. Figure 3-16 gives an overview of the generated methods.

The dialog box is titled "Relation". It contains the following fields and options:

- From:**
 - Class: A
 - Name: A
- Association Type:**
 - ☐ Single
 - ☐ Multi
 - ☒ Static Multi
- Association Properties:**
 - ☒ Aggregation
 - ☐ Critical
 - ☒ Filter
- Implementation:**
 - ☒ Standard
 - ☐ ValueTree
 - ☐ Unique
- To:**
 - Class: B
 - Name: B
 - Member:
- Note:** (A large empty text area)
- Buttons:** OK, Cancel

Figure 3-15: Static multi aggregation between 'A' and 'B'.

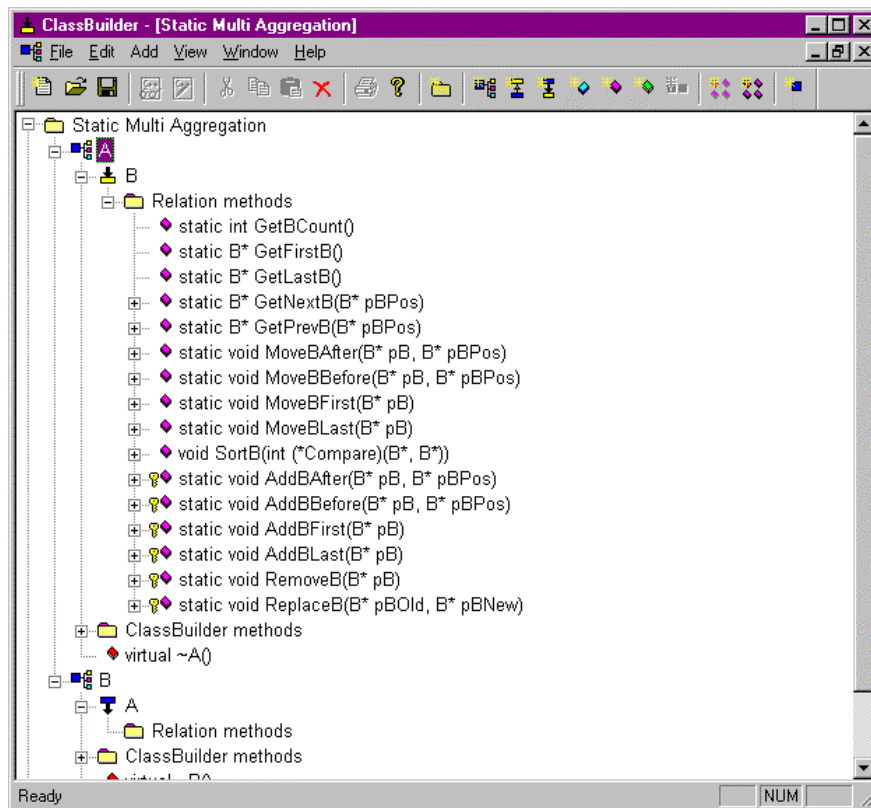


Figure 3-16: The generated methods for a static multi aggregation.

The static multi aggregation has the same relation methods as the static multi association. The difference is that the 'Relation addition methods', 'Relation remove method' and 'Relation substitution method' are now protected methods instead of public. Direct addition to a relation or removal from a relation is prohibited this way. Only the 'Relation interrogation methods' and 'Relation reorder methods' are public and thus directly accessible.

3.10 Naming static multi relations

The naming of the static multi relation works the same as for a multi relation, so see section 3.7 for more details.

4. Dialog box settings

4.1 DataModel

4.2 Class

4.3 Extern Class

It is possible to use already existing classes, together with the classes generated by ClassBuilder. It is not possible to use an existing class as active or passive class of a relation, but it is possible to inherit from an existing class.

4.4 Type

4.5 Inherit

4.6 Relation

4.7 Member

4.8 Member Function (Method)

4.9 Constructor

4.10 Argument

4.11 Group

5. Code editing

6. Other features

6.1 Wrapping methods for members which are classes

6.2 Adding virtual functions to overriding classes.

6.3 Adding 'IsClass' methods.

6.4 Support for multi threaded applications

This tool can be used for multi threaded environments. You can guard a relation from simultaneous access from different threads. To enable this feature for a relation, mark the 'Critical' check box in the relation dialog. Now all methods operating on this relation are guarded. Once a method is started it can't be interrupted by another method from a different thread, operating on the same relation.

It is possible to use the guarding of a relation directly in an application yourself. Suppose in the above example, a complex method on class 'A' using several methods operating on its relation to 'B'. We want this method to be completed without any changes on its relation to 'B' inflicted by another thread. An example of how to act:

```
void A::ComplexMethod()
{
    _criticalSectionB.Enter();

    // Do your Complex stuff

    _criticalSectionB.Leave();
}
```

If used in another context then a method of the active class, the '_criticalSectionXXX' must be preceded with the active class name followed by two double colons. An example of this:

```
void ComplexFunction()
{
    A::_criticalSectionB.Enter();

    // Do your Complex stuff

    A::_criticalSectionB.Leave();
}
```

The include file 'CB_CriticalSection.h' contains the class used for implementing this feature. At this moment this file contains only the implementation for this class for the 'Windows 95' and 'Windows NT' environment. For other platforms this file must be altered. This should be a fairly easy task. The file is included below to show how easy it is to implement.

```
#ifndef CR_CRITICAL_SECTION
#define CR_CRITICAL_SECTION

// This is Windows specific, need to change this for other platforms.
#include <windows.h>
#include <process.h>

class CriticalSection
{
private:
    CRITICAL_SECTION _criticalSection;
```

```
public:
    CriticalSection() { InitializeCriticalSection(&_amp;criticalSection); }
    ~CriticalSection() { DeleteCriticalSection(&_amp;criticalSection); }

    void Enter() { EnterCriticalSection(&_amp;criticalSection); }
    void Leave() { LeaveCriticalSection(&_amp;criticalSection); }
};

#endif
```

7. Writing documentation

8. Serializing your data model

9. Dynamic replacement of objects

Imagine some generic class library with readers from text files to generate a data structure with objects of this class library. With this generic class library a specific application is built. This application requires the addition of extra data members to the existing generic class library. This addition of extra data members can nicely be implemented by inheritance from the generic class library. Now a problem arises. All the readers and other data structure generation code of the generic class library generates only objects of the generic class library. It can not generate the newly inherited objects.

This problem can be solved if the generic class library contains a replace constructor for each class that applications wants to inherit from. This constructor has only one argument and this argument points to the object to replace. It replaces the pointed object by the new object and frees the memory occupied by the old object. Note that the memory is freed and that the old object isn't deleted. The replace constructor acts as a constructor for the new object, but also acts as a destructor for the old object. Since the old object is already destructed, we only have to free its occupied memory. A delete call on the old object would invoke the destructor on this object, while it is already destructed.

The ReplaceConstructorInclude() method supports the easy implementation of such a replace constructor. This method takes care of all relations of the old object and transfer them to the new object. Every object which refer to the old object is updated to refer to the new object. And all objects referenced by the old object is now referenced by the new object. So the new object is placed in the context of the old object.

It does not make sense to replace an object with another object of the same class. The desire is to replace it with an object of an inherited class. This is possible if the class to inherit from has a replace constructor. Just define a constructor for the inherited class which invokes the replace constructor of the base class. Such a constructor, which invokes the replace constructor of its base class is called a promotion constructor. It promotes the old object to a new object of higher order by replacing it. Best to show an example now:

```
class A {
private:
    ReplaceConstructorInclude(A* old);

protected:
    char* _name;

public:
    A(char* name);
    A(A* old); // Replace constructor

    ~A();
};

A::A(char* name)
{
    _name = new char[strlen(name)+1];
    strcpy(_name, name);
}

A::~~A()
{
    delete _name;
}
```

```

A::A(A* old)
{
    ReplaceConstructorInclude(old); // Take care of all relations of old object

    // Do the other members
    _name = old->_name;
    old->_name = NULL;

    // Free the old object.
    old->Free();
}

// Declare a class, which inherits from A
class Ax : public A {
protected:
    int _value;

public:
    Ax(A* old, int value);

    ~Ax() {};
};

Ax::Ax(A* old, int value)
: A(old) // Replace old with the current object
{
    _value = value;
}

// Lets actually use it.
void main()
{
    // Make an A.
    A* a = new A("SomeName");

    // Replace it with an Ax.
    Ax* ax = new Ax(a, 11);
}

```

In this example the details of A's relations have been hidden, but if the relations are made with the ClassBuilder tool they are managed correctly by the `ReplaceConstructorInclude()` method. Note that this method is only included if the 'Replace' option is on in the class dialog.

10. Some 2.0 usage remarks

Since the documentation isn't really updated for the 2.0 release, here some hints about how to use the build in class diagram editor. Read also the "ClassBuilder Problem Report List.doc", this document also describes newly added features.

Note that the double click behaviour has changed. To open the source code of a method, one has to double click while pressing the control key. This has been done to be more consistent with other windows applications and to be consistent with the class diagram editing. Also selecting it with the right mouse button and select 'edit code' via the short cut menu will work.

To make drawings for old designs is simple. Just add a class diagram at the appropriate place (Datamodel, ClassGroup or Class), drag while holding down the ctrl key the desired classes to either an open drawing or to the drawing icon in the tree. Now you can optimise the placement, routing and sizing of the drawing objects and select the desired visible members and methods of the class.

To select members or methods, select 'Select Members & Methods' via the short cut menu and a selection dialog box will pop-up. I know it should also have an entry in the main menu, but I'm thinking about reorganizing the main menu and the toolbars, so any suggestions about a new ordering/grouping of menu commands is welcome.

To change the order of members/methods, select 'Reorder Members & Methods' via the shortcut menu. Or as an alternative select a member/method press an arrow up/down key while holding down the Ctrl key.

To scroll the selected member/method use the arrow up/down key without pressing the Ctrl key.

To select a class, click in the top rectangle where its name is. Multiple select is possible by pressing the shift key while selecting, this will toggle the selected state of the clicked object. Another way of performing a multiple select is by dragging a box around them.

Selected drawing objects can be put on the clipboard and from there included in e.g. Word documents. The generated Rtf documentation will include all drawings.

Relations can not be dragged from one class to another, so change this, just double click on it and the normal dialog box will pop up. The same holds for inheritance. Only the attachment place can be changed.

In addition, the creation of a relation/inheritance isn't graphical, just the normal dialog box will pop up. The defined relation/inheritance will show itself in ALL drawings where both involved classes are on. It has the same automatic behaviour if the add new relation/inheritance is invoked from the tree view. It is however possible to do a selection of two classes and then add a relation or inheritance. For example to create a relation from class 'A' to 'B', do the following: First select class 'A', next select class 'B' while holding down the SHIFT key. Now class 'A' and 'B' are both selected, then press on the 'Add Relation' button in the toolbar. The Relation dialog pop-up, with class 'A' as from class and class 'B' as to class.

The addition of a new class is graphical. After pressing the button in the toolbar a drag box will follow the mouse over the drawing area, click the left button at the desired

place. Then the normal dialog will pop up to inform ClassBuilder about the class name, etc.

It is possible to read the generated Rtf documentation back into ClassBuilder. So you can write out the documentation and then edit it in Word (correct spelling errors etc.) then this changed document can be read back and the note field are updated with the changed information. There is one restriction, the altered text should remain within the bookmarks. Especially when adding text it can be that it is inserted after the bookmark. This is easy to check, when you make the bookmarks visible in word. This can be done in word 7.0 by Tools->Options, select Tab View and check Bookmarks. Now the bookmark start is shown as '[' and the end as ']'. As long as the updated text is between them, the changed text is inserted back into ClassBuilder.

I think the rest of it will explain itself more or less, if not, just try it out! If there are any questions, just ask. Of course I'm interested in any bug found, there isn't a bug small enough and in suggestions for improvements.